Formalizing the mathematics of convexity in Lean

Audrey Xie*
Stanford University
xiea@stanford.edu

Matthew Ho*
Stanford University
matthho@stanford.edu

Ludwig Schmidt†
Stanford University
ludwigsc@stanford.edu

John Duchi†
Stanford University
jduchi@stanford.edu

Abstract

The growing role of artificial intelligence in mathematics makes building formal infrastructure for entire fields increasingly important. We explore a middle-ground between human written formalizations, such as Mathlib, which prioritize understanding how best to formalize mathematics, and AI automated formalization, which can sacrifice semantic faithfulness and reusability for speed. To demonstrate, we formalize the mathematics of convexity from undergraduate textbook results to the research frontier. We hand-author and review formal definitions, statements, and assumptions, while AI agents generate Lean proofs. Proving textbook results helps us determine whether new definitions reflect the intended mathematical concepts before we use them to state research problems. General-purpose AI agents can now formally prove all textbook results we consider, but they fail to resolve problems that were open at the time.

1 Introduction

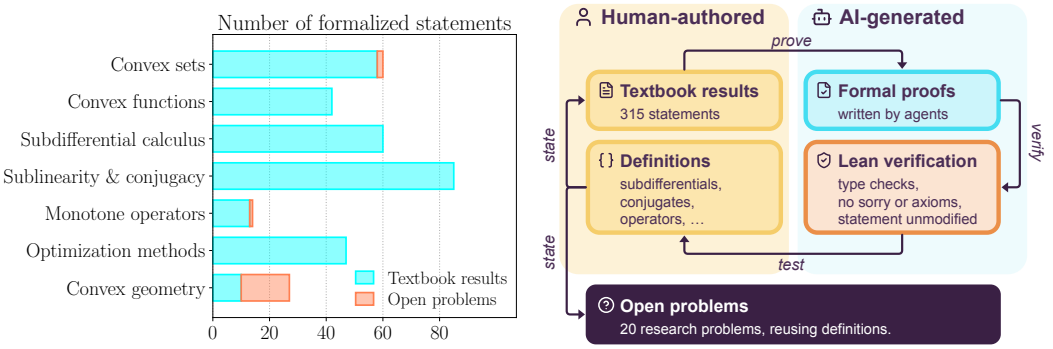
We are moving toward a future where formalization plays an important role in mathematics, either as a first pass check of correctness or as a shared, trusted foundation for collaboration between mathematicians and AI systems. It has consequently become a priority to build out formal infrastructure for large swathes of informal mathematics, starting from what the Lean Mathlib library already covers [de Moura and Ullrich, 2021, The mathlib Community, 2020], and extending all the way to the research frontier.

Mathlib and adjacent projects, such as the human-led formalizations of Fermat's Last Theorem and sphere-packing in eight dimensions, provide not just formalizations but guidance toward understanding how to formalize mathematics; their contributors put a great deal of thought and effort into choosing definitions and constructing modular statements at a high level of generality [Buzzard and Taylor, 2025, Hariharan et al., 2026].

These projects pursue long-term goals that do not always align with the pressures of right-here-and-now use cases of formalization, such as checking journal submissions. On the other hand, projects like Tau Ceti [Tau Ceti Contributors, 2026] and textbook autoformalization pipelines like M2F [Wang et al., 2026] delegate most formalization work to AI, with limited human review. However, automated formalization can introduce subtle drifts in semantic faithfulness, since the Lean type checker may accept a proof even if the formal statement does not capture the intended mathematics [Meek et al., 2026, Bastounis et al., 2026].

*Equal contribution.
†Equal senior authorship.

Preprint.



(a) (b)

Figure 1: Figure 1a shows the distribution of our formalized statements by area. The textbook results focus on areas of convex analysis that Mathlib lacks. Figure 1b illustrates our formalization process. We manually author definitions and statements, then delegate formal proof generation to AI agents. Proving textbook results helps check whether our definitions capture the intended mathematical concepts, allowing us to use them to state research problems.

We explore what a middle ground between fully human and fully automated formalization might look like by building out formal mathematics of convexity from undergraduate textbook results all the way to the research frontier. We focus on convex analysis, optimization, and geometry, because we know these areas well, consider them important, and yet Mathlib's coverage remains limited.

Our approach keeps humans responsible for the mathematical meaning of formal statements while delegating formalization to AI. We hand-author and review definitions, statements, and assumptions to ensure faithfulness to the source material, then use AI models to generate proofs that Lean can check. In total, we formalize 315 results from textbook and research papers, as well as the statements of twenty research problems (Section 2.1). Section 3 describes how we consolidate definitions across sources and detect missing assumptions, and shows some limitations of AI autoformalization.

Section 4 evaluates AI proof generation. Over one year of model releases, general-purpose agents improved from an 11% to 88% solve rate on a subset of 100 textbook tasks. Two current agents, Opus 5.5 and Harmonic's Aristotle, prove all 315 tasks with a budget of two hours. No agents we tested resolve an open problem during our evaluations, although one run improves on the best published packing density bound for Reinhardt's conjecture. Throughout, we document the choices our approach requires, the difficulties we encounter, and how far current models can go with proof generation to help inform future efforts.[1]

2 Preliminaries

2.1 From textbooks to open problems

Mathlib covers the basics of convex sets and convex functions, but it lacks central objects of convex analysis and optimization, such as subdifferentials, Fenchel conjugates, and monotone operators, as well as any optimization methods and their convergence rates. Therefore, for convex analysis foundations, we formalized chapter three of Hiriart-Urruty and Lemaréchal's *Convex Analysis and Minimization Algorithms I* (*CAMA I*) and all of their *Fundamentals of Convex Analysis*, save for the main text of chapter A which overlaps with *CAMA I*. We also formalized exercises from Duchi's EE364M course on the mathematics of convexity [Duchi, 2024].

On the optimization side, we covered most of the textbook *Convex Optimization: Algorithms and Complexity* by Bubeck, convergence rates for the stochastic average gradient method (SAG) [Schmidt et al., 2017], convergence rates for SAGA [Defazio et al., 2014], and chapter 2 of Renegar's *A mathematical view of interior-point methods in convex optimization*.

[1]The project code is available at this link.

For developments in variational analysis and monotone operators, we formalized selected results from chapters 8, 9, 11, and 12 of Rockafellar and Wets' *Variational Analysis* and from Bauschke and Combettes' *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*. In total we formalized 315 textbook results (Figure 1).[2]

Moving beyond textbooks, we formalized statements of twenty research problems. Among others, they include the polynomial Hirsch conjecture, Danzer's problem, and the dimensional Brunn-Minkowski conjecture; see Table 1 for the full list and pointers to their statements.

2.2 Related work

2.2.1 Human-AI collaboration for formalization

Recent projects combine human expertise and AI systems to formalize mathematical results, with different divisions of AI and human labor. Human contributions to proof planning as well as the design and review of formal statements have supported AI-assisted formalizations of sphere packing and the Poincaré conjecture [Hariharan et al., 2026, Zhang et al., 2026b]. FormalFlow delegates authorship of formal definitions, statements, and proofs to agents, while humans provide oversight and review the final theorem statement and its dependent definitions [Lu et al., 2026]. In software verification, ARTEMIS [Mendoza et al., 2026], PICK [Prasad et al., 2026], and nl2spec [Cosler et al., 2023] incorporate human feedback to assess the faithfulness of AI-generated specifications and correct misformalizations.

Other projects use human-AI collaboration for textbook formalization and the construction of reusable mathematical libraries in statistical learning theory, probability, and computational economics [Zhang et al., 2026a, Deng and Shum, 2026, Bei et al., 2026]. Following a similar philosophy, we build reusable infrastructure for convex analysis, optimization, and geometry, as convex mathematics is an area used in engineering disciplines we are familiar with. The closest existing library in this area is optlib, which contains convergence proofs for several first-order methods, but it contains no stochastic methods, nor does it systematically formalize convex analysis [Li et al., 2025]. We formalize from textbook foundations to research problems, with humans authoring definitions and theorem statements and AI generating proofs.

2.2.2 Autoformalization

Many recent formalization efforts have used large language models (LLMs) to autoformalize both statements and proofs of mathematical results. Earlier work has shown that machine learning models are able to translate natural language statements to formal languages [Wang et al., 2018, Wu et al., 2022]. Weng et al. [2025] survey progress in autoformalization and formal proof generation. Recent work has demonstrated that agentic systems can autoformalize at scale; agentic pipelines for textbook formalization include one for numerical analysis [Meek et al., 2026] and M2F, a general pipeline for formalizing textbooks [Wang et al., 2026], while pipelines for paper formalization include MerLean for quantum computation [Ren et al., 2026] and EconCSLib for economics and computation [Garg, 2026]. Numerous agents working in parallel have formalized complete proofs of major results, including Fermat's Last Theorem [Anthropic, 2026a] and finite-time blowup for the forced Navier–Stokes equations [OpenAI, 2026l]. Pipelines that also delegate statement formalization to AI offer an alternative to our proposed allocation of work for building formal mathematical libraries.

3 Formalization

Formalizing new material requires making choices about how closely to follow the textbook presentation and which assumptions to make explicit. We now describe these choices in the context of convex analysis and optimization.

3.1 Definitions at the right level of generality

To formalize convex mathematics from undergraduate-level textbooks to research-level problems, we need definitions and structures that work for a range of statements, from simple to complex. We

[2] See Appendix I for the full list of textbook results.

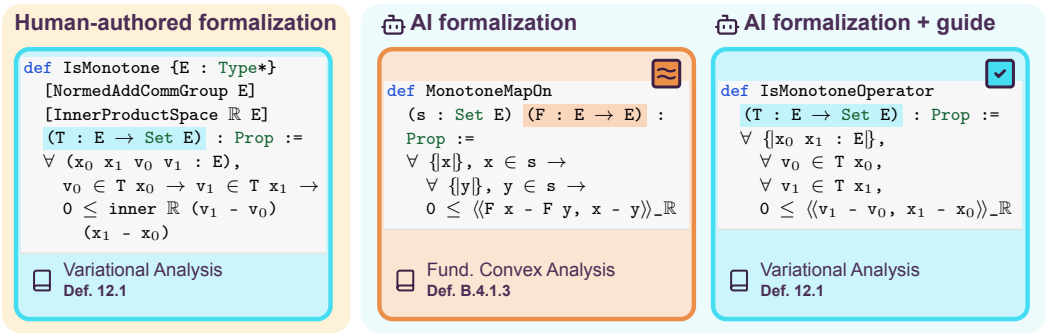


Figure 2: Comparison of human-authored and AI-generated formalizations of monotonicity by Fable 5.1. The agent formalizes the single-valued definition of monotonicity from *Fundamentals of Convex Analysis* and overlooks the set-valued definition from *Variational Analysis*. Providing a detailed guide containing definitional choices leads the agents to formalize the more general definition.

use definitions from Mathlib whenever they correspond to the intended mathematics. This allows us to build on existing results and focus on formalizing the additional definitions necessary to state our material. Our goal with these new definitions is to state them at a level of generality that enables reuse across the selected textbooks, rather than their most general form.

3.1.1 Functions and operators

Textbooks covering the same area of mathematics often introduce similar concepts at different levels of generality, so formalizing each of these sources separately can result in several definitions that actually correspond to the same concept. Building shared formal infrastructure therefore involves gathering these definitions and deciding how they relate and which to select. In instances where one generalizes another, we use the more general definition.

For example, we formalize conjugacy for functions taking values in the extended reals, $f : E \to \mathbb{R} \cup \{\pm\infty\}$, which follows the version in *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*.

Definition 3.1 (*Convex Analysis and Monotone Operator Theory in Hilbert Spaces*, Definition 13.1) *Let* $f : \mathcal{H} \to [-\infty, +\infty]$. *The* conjugate *(or* Fenchel conjugate*, or* Legendre transform*, or* Legendre-Fenchel transform*) of* f *is*

$$f^* : \mathcal{H} \to [-\infty, +\infty] : u \mapsto \sup_{x \in \mathcal{H}} \left(\langle x \mid u \rangle - f(x) \right), \tag{1}$$

and the biconjugate *of* f *is* $f^{**} = (f^*)^*$.

This also makes defining biconjugates convenient as both the input and output functions have the same codomain, thus avoiding the type conversions that plagued our earlier formalization following the *Fundamentals of Convex Analysis* definition for functions mapping to $\mathbb{R} \cup \{+\infty\}$. To give another example, the notion of monotonicity in *Fundamentals of Convex Analysis* is a special case of the set-valued definition in *Variational Analysis*.

Definition 3.2 (*Variational Analysis*, Definition 12.1) *A mapping* $T : \mathbb{R}^n \rightrightarrows \mathbb{R}^n$ *is called* monotone *if it has the property that*

$$\langle v_1 - v_0, x_1 - x_0 \rangle \geq 0 \quad \text{whenever} \quad v_0 \in T(x_0), v_1 \in T(x_1) \tag{2}$$

and strictly monotone *if this inequality is strict when* $x_0 \neq x_1$.

Simply consider the single-valued function as mapping to the singleton set. We therefore formalize and use the set-valued version throughout.

We find that agents can miss these cross-textbook generalizations. We gave GPT-6 Astra, Fable 5.1, and Opus 5.5 access to all sources and asked them to formalize Theorem 4.1.4 in Chapter B of *Fundamentals of Convex Analysis* at a "suitable level of generality," noting that the result

```
noncomputable def projSubgradientStep {n : ℕ}
  (𝒳 : Set (EuclideanSpace ℝ (Fin n)))
  (x : EuclideanSpace ℝ (Fin n))
  (g : EuclideanSpace ℝ (Fin n))
  (η : ℝ)
  (h𝒳_nonempty : Set.Nonempty 𝒳)
  (h𝒳_compact : IsCompact 𝒳)
  (h𝒳_convex : Convex ℝ 𝒳) : EuclideanSpace ℝ (Fin n) :=
  let y_next := x - η • g
  pC y_next 𝒳 h𝒳_compact.isClosed h𝒳_convex h𝒳_nonempty

def IsProjectedSubgradientSequence {n : ℕ}
  (𝒳 : Set (EuclideanSpace ℝ (Fin n)))
  (f : EuclideanSpace ℝ (Fin n) → ℝ)
  (x : ℕ → EuclideanSpace ℝ (Fin n))
  (g : ℕ → EuclideanSpace ℝ (Fin n))
  (η : ℕ → ℝ)
  (h𝒳_nonempty : Set.Nonempty 𝒳)
  (h𝒳_compact : IsCompact 𝒳)
  (h𝒳_convex : Convex ℝ 𝒳) : Prop :=
  (∀ t, x (t + 1) = projSubgradientStep 𝒳 (x t) (g t) (η t) h𝒳_nonempty h𝒳_compact h𝒳_convex) ∧
  (∀ t, InDomIsSubgradientAt 𝒳 f (x t) (g t))
```

Figure 3: Formalization of the projected subgradient descent method. The update step is formalized as `projSubgradientStep` and makes no assumption on the vector g. Instead, the assumption that every vector g_t is a subgradient of f at iterate x_t gets deferred to `IsProjectedSubgradientSequence`.

would be part of a project formalizing the entire corpus. This theorem characterizes convexity of a differentiable function by the monotonicity of its gradient, a single-valued function. All three agents chose to formalize the single-valued definition of monotonicity (Figure 2). When subsequently asked to formalize Theorem 20.25 of *Convex Analysis and Monotone Operator Theory*, which shows convexity implies monotonicity of a function's set-valued subdifferential, all agents introduced a separate set-valued definition without revising the previous theorem, leaving two definitions of monotonicity in the project.

Although each definition was sufficient for its respective theorem, the agents missed this opportunity for consolidation. Their formalizations aligned with ours only when explicitly guided on the desired formalization choices, which supports our choice to keep definitions human-authored when building reusable formalizations.[3]

3.1.2 Optimization methods

Most optimization methods consist of an initialization and rule for producing the next iterate. Formalizing these descriptions requires choosing how to represent a trajectory of the method. One option is to construct the iterates recursively from initial conditions via explicit n-fold compositions, while a second, perhaps more natural for most human readers, provides a single-step update with a set of "invariants" that any valid iterate trajectory of the method satisfies. We take the latter approach, as it directly captures the update rules, oracle conditions, and randomness assumptions used in convergence analysis, allowing us to state convergence rates without explicitly constructing trajectories. We use this approach to formalize the deterministic and stochastic algorithms in Bubeck's *Convex Optimization: Algorithms and Complexity*. In addition, we also formalize SAG and SAGA. We define predicates that take sequences of points and step sizes and specify how successive points relate via the method's update rule.

For example, projected subgradient descent is defined as, for $t \geq 1$,

$$y_{t+1} = x_t - \eta g_t, \quad g_t \in \partial f(x_t), \tag{3}$$

$$x_{t+1} = \Pi_{\mathcal{X}}(y_{t+1}). \tag{4}$$

We define the update rule `projSubgradientStep`, shown in Figure 3, to accept any vector g with no assumption that g is a subgradient.

[3] See Appendix G for additional examples.

```
def IsSubgradientOracle {n : ℕ}
  (𝒳 : Set (EuclideanSpace ℝ (Fin n)))
  (f : EuclideanSpace ℝ (Fin n) → ℝ)
  (G : EuclideanSpace ℝ (Fin n) → EuclideanSpace ℝ (Fin n)) : Prop :=
  ∀ z ∈ 𝒳, InDomIsSubgradientAt 𝒳 f z (G z)

def IsBlackBoxSpanProcedure {n : ℕ}
  (𝒳 : Set (EuclideanSpace ℝ (Fin n)))
  (G : EuclideanSpace ℝ (Fin n) → EuclideanSpace ℝ (Fin n))
  (x : ℕ → EuclideanSpace ℝ (Fin n)) : Prop :=
  (x 0 = 0) ∧
  (∀ s, x s ∈ 𝒳) ∧
  (∀ s, x (s + 1) ∈ Submodule.span ℝ ((fun j => G (x j)) '' Set.Iic s))
```

Figure 4: A black-box procedure is formalized as `IsBlackBoxSpanProcedure`, where given a subgradient oracle G, the procedure produces an iterate in the linear span of the previous oracle outputs. This representation is useful in formalizing various oracle complexity lower bounds, where one exhibits slow convergence of any black-box procedure for a chosen function f and oracle G.

The predicate `IsProjectedSubgradientSequence` then requires each successive iterate to follow from `projSubgradientStep` and each g_t be a subgradient of f at x_t. These conditions "constrain" the given sequence of iterates, vectors, and step sizes into a trajectory of the projected subgradient descent method. Using this format, we can describe and prove the convergence rates of many common optimization methods.

In addition to upper bounds on convergence rates, we also wish to establish lower bounds that characterize fundamental limits of first-order black-box algorithms. Following Bubeck, we consider black-box procedures defined, for $t \geq 0$,

$$x_{t+1} \in \mathrm{Span}\{g_1, \ldots, g_t\} \tag{5}$$

and $x_1 = 0$. Since a function may admit multiple subgradients at each point, and a lower bound construction depends on which subgradients the oracle returns, formalization must distinguish the algorithm's choice of iterates from the oracle's choice of subgradients. As Figure 4 shows, we represent the oracle as a function G where $G(z)$ is a valid subgradient of f for every $z \in \mathcal{X}$.

Our predicate `IsBlackBoxSpanProcedure` takes as input the constraint set $\mathcal{X}$, the oracle G, and an iterate sequence x, then requires successive iterates to reside in the linear span of the previous oracle outputs. This extends our sequence-based representation of optimization methods while allowing the lower bound construction to choose a function f and oracle G, so the statements can then quantify over all trajectories satisfying `IsBlackBoxSpanProcedure`. This formalization allows us to correctly state Theorems 3.13 and 3.14 in *Convex Optimization: Algorithms and Complexity* which, to our knowledge, is the first formalization of these first-order black-box oracle lower bounds.

3.2 Detecting and filling formalization gaps

Even after formalizing all definitions needed to state a textbook result, gaps in the formalization may still remain. Textbooks often leave some assumptions implicit, relying on the surrounding discussion, standard mathematical conventions, and other conditions considered "obvious" to the reader to complete the statement. Formalization requires identifying these assumptions and making them explicit, since omission can make the resulting statement false. In this section, we describe the methods we used to detect these gaps and fill them with assumptions that align with the intended mathematical meaning.

3.2.1 Detecting formalization gaps

To identify missing assumptions, we first read the surrounding context and then authored an initial formal version of the textbook statement. We then prompted a theorem-proving agent, either Claude Code [Anthropic, 2025a] or Aristotle [Achim et al., 2025], to generate a formal proof. Since a proof that passes Lean's type checker does not mean that a statement is faithful, we checked whether each successful proof held for unintended reasons. For example, a statement could be vacuously true due to a missing condition. Other attempts produced counterexamples or returned an incomplete proof, which helped us identify potential gaps.

Table 1: Each formalized research problem with a link to its natural-language statement.

Problem	App.	Problem	App.
Bang's affine plank problem	B.1	Logarithmic Brunn–Minkowski conjecture	B.11
Borsuk's partition conjecture ($n \leq 62$)	B.2	Mahler's conjecture, general position	B.12
Klee's Chebyshev set problem	B.3	Mahler's conjecture, symmetric case	B.13
Danzer's problem	B.4	Order-six Drew–Johnson–Loewy conjecture	B.14
Dimensional Brunn–Minkowski conjecture	B.5	Petty's projection conjecture	B.15
Generalized Lax conjecture	B.6	Polynomial Hirsch conjecture	B.16
Hadwiger's covering conjecture	B.7	Fixed points in reflexive Banach spaces	B.17
Kalai's 3^d conjecture	B.8	Reinhardt's conjecture	B.18
Kannan–Lovász–Simonovits conjecture	B.9	Shannon capacity of the 7-cycle	B.19
Kneser–Poulsen conjecture	B.10	Sharp symmetric slicing conjecture	B.20

When we identified a missing assumption, we considered whether it would be reasonable given the surrounding context. If so, then we added it to the statement and repeated the detection process. Our goal was to recover a reasonable, non-contradictory set of assumptions under which the textbook result holds, and we typically achieved this within a few iterations.

3.2.2 Filling formalization gaps

We used this process extensively to identify the implicit measurability and integrability assumptions in stochastic optimization results from *Convex Optimization: Algorithms and Complexity*. Stochastic mirror descent provides an illustrative example. At a query point x, the stochastic oracle outputs $\tilde{g}(x)$ satisfying,

$$\mathbb{E}[\tilde{g}(x) \mid x] \in \partial f(x). \tag{6}$$

For our formalization to correspond to the conditional expectation, we additionally require that for every t, x_t is measurable, $f(x_t)$ and $f(\bar{x}_t)$ are a.e. measurable, and $||g_t||_*^2$ is integrable. We reused this set of assumptions across all the stochastic optimization bounds. This set is consistent and sufficient for the bounds to hold, but we want to note that it is not the most minimal set.

3.3 Formalizing open problems

We selected twenty research problems spanning convex analysis, optimization, and geometry, which we list in Table 1. We give their natural-language statements and relevant sources in Appendix B. To perform this selection, we first curated research problems with AI assistance and then we manually reviewed each selected problem for significance, relevance, and formalization requirements. We then performed adversarial literature searches to confirm the selected problems remained open. Since our evaluation focuses on sustained agent work, we used GPT-5.6 Sol Pro to check whether a frontier model could already resolve the problem and removed candidate problems that were resolved by GPT-5.6 Sol Pro.

For each problem, we formalized statements according to three criteria in decreasing order of precedence: (i) apparent-at-a-glance equivalence to the standard conjecture, (ii) minimal introduction of new definitions, and (iii) the greatest practical level of generality. The first criterion makes statement faithfulness easier to check by avoiding reliance on nontrivial equivalence arguments. The second criterion limits the number of new definitions that require checking. Unlike textbook definitions used across many proved results, definitions introduced for individual conjectures offer fewer opportunities for errors to surface through reuse.

In practice, stating the open problems required fifty-six definitions, twenty of which we reused from the textbook development (thirteen verbatim and seven as specializations to Euclidean space). The remaining definitions, such as packing density, hyperbolicity cones, and Shannon capacity, do not appear in the selected textbooks and thus received additional review.

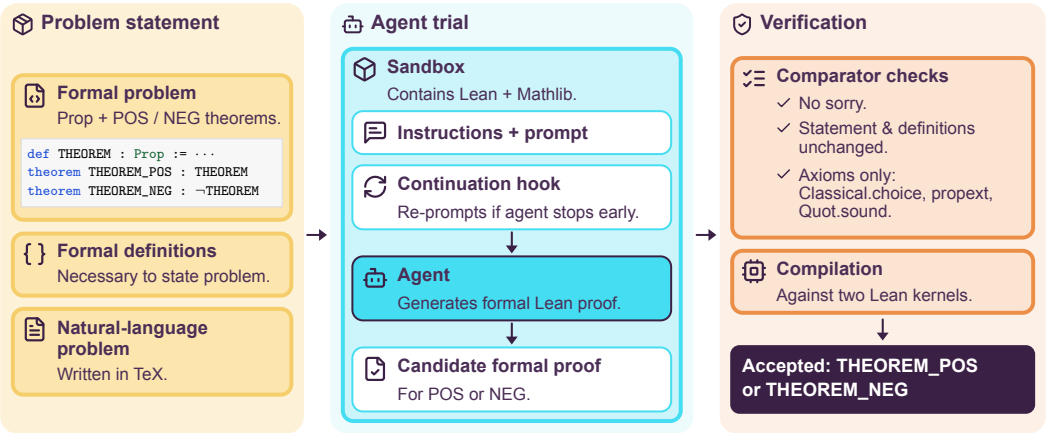


Figure 5: In the open problem setting, agents receive the Lean statement, associated formal definitions, and the natural-language statement. They must produce a formal proof resolving the problem in either the positive or negative direction. The generated proof is then checked using Comparator against two independent Lean kernel implementations.

4 Proof generation

After formalizing and checking the statements, we evaluated how far current models could go in generating proofs. The textbook tasks ask agents to prove known results in Lean, and they also serve as an integration test for our definitions, showing that the results that should hold can be proved. The open problems instead require new mathematical work as well as a formal proof. We found that a general-purpose agent harness such as Claude Code paired with a frontier model could autonomously produce formal proofs for all of our selected textbook results. However, the agent did not resolve any of the open problems within the allotted budget.

4.1 Experimental setup

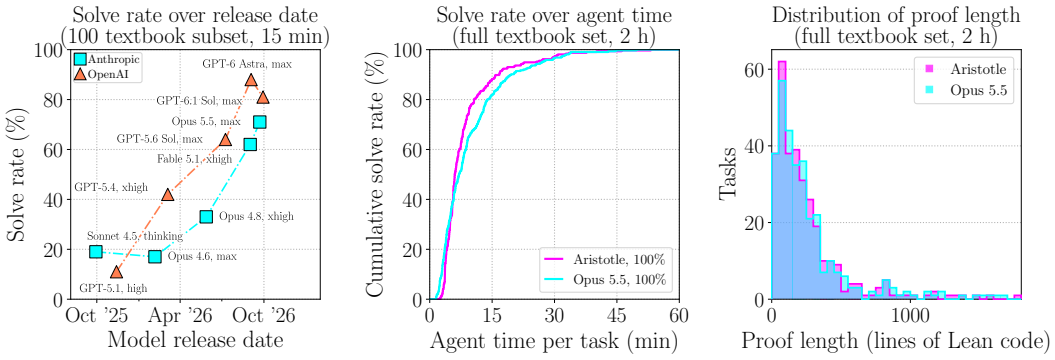
We conducted textbook evaluations with time limits of fifteen minutes and two hours, as well as an evaluation on the open problems. Each task contained a Lean theorem statement as well as all the definitions needed to state it. The open problems also included a TeX document containing the natural-language problem statement.

Fifteen minute textbook evaluation. We evaluated frontier models from Anthropic and OpenAI released between September 2025 and September 2026 on a subset of 100 textbook tasks, with a fifteen minute time limit per task. We omitted the time limit from the instructions because mentioning it caused many of the older agents to terminate early. We paired each model with its corresponding agent harness, Claude Code [Anthropic, 2025a] or Codex [OpenAI, 2025a], selecting the earliest harness version following the model's release date.[4]

Two hour textbook evaluation. We evaluated Harmonic's Aristotle agent [Achim et al., 2025] and Claude Code with Opus 5.5 [Anthropic, 2026e] on the full set of 315 textbook tasks, with a two hour time limit per task. Unlike the fifteen minute evaluation, we explicitly stated the time limit in the instructions.

Open problem evaluation. Open problems have either a positive or negative resolution. We therefore defined each conjecture as a `Prop`, and then routed it to two theorems, `{THEOREM}_POS` and `{THEOREM}_NEG`, asking the agent to prove either the proposition or its negation (Figure 5). We supplied an agent-specific prompt,[5] selecting the prompt with which the agent produced the most natural-language mathematical reasoning among candidate prompts.

[4] See Appendix E for the full list of frontier models and harness versions.
[5] See Appendix H.2.2 for the full prompt.



(a) (b) (c)

Figure 6: General purpose agents have improved in formal proof generation, from an 11% to an 88% solve rate under a 15 minute time limit, over the course of a year (Figure 6a). Figure 6b shows the cumulative solve rate for the textbook tasks. Both Aristotle and Claude Code with Opus 5.5 prove all of the textbook tasks in under two hours, and their generated formal proofs range from a few lines to over 1,500 lines long (Figure 6c).

We also implemented a continuation hook that automatically prompted agents to resume whenever they stopped without a successful submission and still had time or budget remaining. We ran each problem at budgets of $100 and $1,000, both with a 96 hour time limit. The two budgets lets us compare how agents plan under different resource constraints. Since the time limit was intended to terminate idle runs rather than constrain agent work, only the budget was mentioned in the instructions.

4.1.1 Agent environment

We used the Harbor framework [Harbor Framework Team, 2026] for each evaluation. Every Claude Code and Codex agent ran in a sandbox with Mathlib installed. The fifteen minute evaluations used Lean v4.21.0, which was released before all of the models' training cutoff dates, while the two hour evaluations used Lean v4.28.0, which was released before the cutoff date of Opus 5.5. The task statement and required definitions were the only additional Lean files present in the sandbox. We supplied task instructions outlining both the task and the success criteria to each agent. For the open problem evaluations, the sandbox also contained the TeX problem statement, and we additionally supplied the agent prompt in Appendix H.2.2.

Aristotle ran through its API, which bundles both the agent and its execution environment. Therefore, we provided the same instructions, formal statements, and definitions, but without the sandbox.

4.1.2 Proof verification

Across all three settings, we used Comparator [Lean FRO, 2025] to guard against verification exploits. Comparator requires successful proofs to contain no instances of `sorry`, preserve the theorem statement and definitions, and use no axioms beyond `Classical.choice`, `propext`, and `Quot.sound`. It also checks proofs with nanoda [Bailey, 2020], an independent implementation of Lean's kernel written in Rust. Checking with both kernels is meant to catch exploits of a bug in either implementation. While this still may not be bulletproof, it decreases the probability of an exploit escaping detection.

4.2 Evaluations on textbook tasks

Our textbook tasks evaluated whether agents could produce formal proofs of known results. We found that agents showed substantial improvement in these capabilities over the course of one year. Under the fifteen minute time constraint, the success rate rose from 11% for GPT-5.1 and 19% for Sonnet 4.5, both released during the latter half of 2025, to 88% and 71% for GPT-6 Astra and Opus 5.5, both released in September 2026 (Figure 6a).

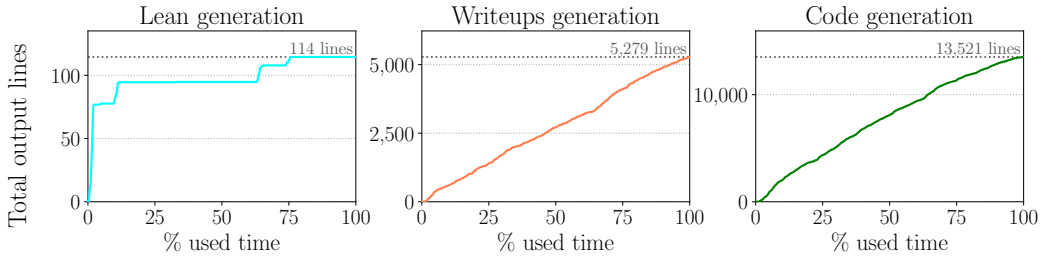


Figure 7: The total number of lines of Lean, natural language writing, and other code generated over the course of the $1,000 budget open problems trials, averaged over the trials. The majority of Lean code is generated at the beginning of each trial. On the other hand, natural language writing and reasoning as well as code is produced throughout the trial, with code generation reducing towards the end of each trial.

With a two hour time limit, both Aristotle and Claude Code with Opus 5.5 proved every textbook result (in fact in under one hour (Figure 6b)). The generated proofs ranged from a few lines long, from applying existing Mathlib results, to more than 1,500 lines for results from the later chapters of *Variational Analysis* (Figure 6c). We note that each formal statement was named after its textbook source. We intentionally chose not to obscure the origin of each statement, since textbook tasks measure how far agents can go with proof generation rather than proof discovery. We observed that in 28% of the trials the agents referred to the exact textbook in their transcripts. For example, on Proposition 8.32 of *Variational Analysis*, the agent wrote, "This looks like a well-known result from Rockafellar-Wets' Variational Analysis, specifically Proposition 8.32."

In three trials, Claude Code with Opus 5.5 used Python for numerical or symbolic computation. In two of the three, the results of the code were directly used in the final Lean proofs. For the Newton decrement bound (Theorem 5.4 of *Convex Optimization: Algorithms and Complexity*), the agent used SymPy to derive $|D^3 f[a,u,u]| \leq 2||a||_x||u||_x^2$ from self-concordance. This is a known result, but it is unavailable in Mathlib.

For the strongly convex SAG rate, the agent used SymPy to find an explicit polynomial certificate for the one-step decrease. This replaces the original proof's computer-assisted verification with an argument entirely verified by Lean.

4.3 Evaluations on open problems

We then evaluated agents on the twenty open problems which we formalized in Lean. Fable 5.1 was unable to resolve any of the problems under the given budgets. Manual inspection of trajectories revealed that agents typically began by checking the faithfulness of the Lean formalizations to the open problems, either by testing edge cases or inspecting Mathlib libraries.

Agents also searched the web for relevant work and progress on the problem after their knowledge cutoff. We found evidence of formalization checks in 39 of 40 trajectories and literature searches in all 40. None of the checks identified a loophole that would allow the formal statement to be resolved without solving the intended problem.

To analyze how agents allocated their effort, we classified writes by file type: Markdown and LaTeX as "writeup", Lean as "Lean", and Python, C, and shell scripts as "code". Agents spent most of their time on natural-language mathematics, with minimal Lean generation beyond initial verification (Figure 7). This is consistent with our prompt, which instructs agents to prioritize mathematical reasoning before formalization.[6]

4.3.1 Intermediate progress metrics

Even though no trajectory produced a complete solution, we examined whether Fable 5.1 produced any new, intermediate results. In the $1,000 Reinhardt conjecture run, it obtained a packing density lower bound of 0.8950112, which is better than the best published bound we found, 0.892656 [Tammela,

[6]See Appendix H.2.2 for the full prompt.

1970]. We formalized this new lower bound in Lean after the run, verifying agent mathematical progress was legitimate.[7]

We found that agents explicitly reasoned about their remaining budget when planning future work. In at least 19 of the 20 trajectories with a $100 budget, agents mentioned their budget at least once when deciding which next steps to pursue. For example, the agent working with a $100 budget for the nonsymmetric Mahler conjecture stated:

> This is realistically thousands of lines of Lean and months of effort, which rules out attempting the positive direction given the session's budget — so I need to consider whether the conjecture could instead be false… Given there's no partial credit… I think the most valuable path forward is pursuing the four-dimensional (and general n) question of whether the Chen–Li–Xi–Xu approach extends. I'll write exhaustive search code…

After our evaluation and during our analysis, eight of the twenty open problems were resolved [OpenAI, 2026e,f,g,h,i,j,k, Song and Zhang, 2026]. For the seven resolved by OpenAI, we compared Fable 5.1's work to the published solutions by manually examining all 14 trajectories (seven problems ran with budgets of $100 and $1000). Every trajectory explored both directions of resolution, as instructed, and in 13 of 14 cases, the agent focused more on the direction ultimately shown to be correct. The exception was the $1,000 run for the generalized Lax conjecture, where the agent worked on proving the conjecture and the conjecture was disproved.

We observed that literature searches appeared to influence which directions agents pursued. For example, in the $100 generalized Lax run, the agent worked more towards finding a counterexample. However, the $1,000 agent discovered a preprint published after the $100 run establishing the conjecture for strictly hyperbolic polynomials [Kummer and Netzer, 2026]. The $1,000 run subsequently focused more on trying to prove the conjecture rather than disprove it.

5 Conclusion

We formalized the mathematics of convexity in Lean, combining human-authored definitions and theorem statements with AI-generated formal proofs to establish a middle ground between fully human-written and fully automated formalization. Our autoformalization examples show how current agents can overlook opportunities for definitional reuse across sources, highlighting the value of human guidance in building shared mathematical infrastructure. We show that general-purpose AI agents can generate formal proofs for all 315 selected textbook and paper results, which suggests that essentially, as soon as a human-language proof is available in an area Mathlib covers, raw lean formalization needs little human involvement. For open problems, detailed prompting and a continuation mechanism enabled sustained mathematical exploration throughout the agents' trajectory. Although agents did not resolve any of the twenty problems, they produced a novel intermediate result and provided insight into how agents pursue questions at the research frontier.

This work suggests a few future directions. First, systematic evaluations could assess agents' judgment in developing reusable definitions and faithfully formalizing natural language statements and proofs. A related challenge is how to use formalization to identify gaps in the original argument, rather than merely establishing a theorem's correctness. Lastly, closer human-AI collaboration could connect the development of Lean libraries with mathematical exploration. Agents' investigations could inform new human authored definitions and intermediate statements, and the resulting formal infrastructure could support further mathematical discovery.

Acknowledgments and Disclosure of Funding

We thank Gregory Xie, Eric Ma, and Anand Rao Tadipatri for helpful feedback on the paper. MH is supported by the National Science Foundation Graduate Research Fellowship Program under Grant No. DGE-2146755. AX, JD, and MH are partially supported by the Office for Naval Research under Grant N00014-22-1-2669. We acknowledge the NSF AI Institute for Foundations of Machine Learning (IFML), Open Philanthropy, Microsoft Grant in Customer Experience Innovation, Anthropic,

[7]The Lean v4.33 formalized proof is available at this link, verified by Comparator.

and Apple. We also thank Harmonic, Anthropic, and Daytona for providing API access and credits that supported this research.

AI Usage Disclosure

Research. We hand-authored all Lean definitions and theorem statements and only used AI theorem proving agents and general purpose agents (Aristotle agent, Claude Code, Codex) for assistance with checking human-written formalizations for correctness. We used LLM agents to produce code scripts throughout the project, such as for plot generation and for writing scripts profiling the type of files Fable 5.1 produced during its evaluation. In the analysis of Fable 5.1 agent trajectories, we used LLM agents to build a website interface to make it easier for us to analyze trajectories and to highlight selected portions of trajectories that are more likely to be relevant to aid our analysis. We used agent assistance to perform literature search for open problems and to filter out easier candidates.

Writing. We used Claude Code and Codex to aid literature search and as aids for LaTeX formatting and citation generation of sources we collected. All citations were checked against the corresponding original source. All prose was written by the authors.

References

Tudor Achim, Alex Best, Alberto Bietti, Kevin Der, Mathïs Fédérico, Sergei Gukov, Daniel Halpern-Leistner, Kirsten Henningsgard, Yury Kudryashov, Alexander Meiburg, Martin Michelsen, Riley Patterson, Eric Rodriguez, Laura Scharff, Vikram Shanker, Vladmir Sicca, Hari Sowrirajan, Aidan Swope, Matyas Tamas, Vlad Tenev, Jonathan Thomm, Harold Williams, and Lawrence Wu. Aristotle: IMO-level automated theorem proving. *CoRR*, abs/2510.01346, 2025. doi: 10.48550/ARXIV.2510.01346. URL https://doi.org/10.48550/arXiv.2510.01346.

A. D. Aleksandrov. Mean values of the support function. *Doklady Akademii Nauk SSSR*, 172(4): 755–758, 1967. URL https://www.mathnet.ru/eng/dan32847. In Russian.

Anthropic. Claude Code, 2025a. URL https://github.com/anthropics/claude-code. Agentic coding tool.

Anthropic. System Card: Claude Sonnet 4.5. Technical report, Anthropic, September 2025b. URL https://www.anthropic.com/claude-sonnet-4-5-system-card.

Anthropic. Formalizing Fermat's Last Theorem. https://www.anthropic.com/research/formalizing-fermats-last-theorem, 2026a. Lean formalization available at https://github.com/anthropics/fermats-last-theorem.

Anthropic. System Card: Claude Fable 5.1 and Claude Mythos 5.1. Technical report, Anthropic, September 2026b. URL https://www.anthropic.com/claude-fable-5-1-mythos-5-1-system-card.

Anthropic. System Card: Claude Opus 4.6. Technical report, Anthropic, February 2026c. URL https://www.anthropic.com/claude-opus-4-6-system-card.

Anthropic. System Card: Claude Opus 4.8. Technical report, Anthropic, May 2026d. URL https://www.anthropic.com/claude-opus-4-8-system-card.

Anthropic. System Card: Claude Opus 5.5. Technical report, Anthropic, September 2026e. URL https://www.anthropic.com/claude-opus-5-5-system-card.

Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics. *CoRR*, abs/2302.12433, 2023. doi: 10.48550/ARXIV.2302.12433. URL https://doi.org/10.48550/arXiv.2302.12433.

Christopher Bailey. nanoda. https://github.com/ammkrn/nanoda_lib, 2020.

Keith Ball. Logarithmically concave functions and sections of convex sets in $\mathbb{R}^n$. *Studia Mathematica*, 88(1):69–84, 1988. doi: 10.4064/sm-88-1-69-84. URL https://matwbn.icm.edu.pl/ksiazki/sm/sm88/sm8817.pdf.

Ram Prakash Bambah and Alan C. Woods. On a problem of Danzer. *Pacific Journal of Mathematics*, 37(2):295–301, 1971. URL https://msp.org/pjm/1971/37-2/pjm-v37-n2-p02-s.pdf.

Thøger Bang. A solution of the "plank problem". *Proceedings of the American Mathematical Society*, 2(6):990–993, 1951. doi: 10.1090/S0002-9939-1951-0046672-4. URL https://doi.org/10.1090/S0002-9939-1951-0046672-4.

Alexander Bastounis, Fabian Circelli, and Anders C Hansen. Navier-Stokes lost in translation: Why Lean verification of AI autoformalisation does not guarantee correct natural language proofs. *arXiv preprint arXiv:2610.08144*, 2026.

Heinz H. Bauschke and Patrick L. Combettes. *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*. CMS Books in Mathematics. Springer, second edition, 2017. ISBN 978-3-319-48311-5. doi: 10.1007/978-3-319-48311-5.

Xiaohui Bei, Jiajun Ma, Zhan Jing, Hongfei Fu, and Zhihao Gavin Tang. EconCSLib: A Lean library for computational economics and AI-assisted research. *CoRR*, abs/2606.16144, 2026. doi: 10.48550/ARXIV.2606.16144. URL https://doi.org/10.48550/arXiv.2606.16144.

S. G. Bobkov. Isoperimetric and analytic inequalities for log-concave probability measures. *The Annals of Probability*, 27(4):1903–1921, 1999. doi: 10.1214/aop/1022874820. URL https://www-users.cse.umn.edu/~bobko001/papers/1999_AOP_Isop.pdf.

Károly J. Böröczky, Erwin Lutwak, Deane Yang, and Gaoyong Zhang. The log-Brunn–Minkowski inequality. *Advances in Mathematics*, 231(3–4):1974–1997, 2012. doi: 10.1016/j.aim.2012.07.015. URL https://doi.org/10.1016/j.aim.2012.07.015.

Karol Borsuk. Drei Sätze über die n-dimensionale euklidische Sphäre. *Fundamenta Mathematicae*, 20:177–190, 1933. doi: 10.4064/fm-20-1-177-190. URL https://doi.org/10.4064/fm-20-1-177-190.

Thomas Browning, Katerina Hristova, David Ledvinka, Justus Springer, Hang Lu Su, Jack McKoen, and Kevin Buzzard. A dataset of formalised theorem statements from the Annals of Mathematics, 2026. URL https://arxiv.org/abs/2610.18301.

Sébastien Bubeck. Convex optimization: Algorithms and complexity. *Found. Trends Mach. Learn.*, 8(3–4):231–357, November 2015. ISSN 1935-8237. doi: 10.1561/2200000050. URL https://doi.org/10.1561/2200000050.

Kevin Buzzard and Richard Taylor. FLT: A formalisation of Fermat's Last Theorem in Lean. https://github.com/ImperialCollegeLondon/FLT, 2025.

Luoxin Chen, Jinming Gu, Liankai Huang, Wenhao Huang, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Kaijing Ma, Cheng Ren, Jiawei Shen, Wenlei Shi, Tong Sun, He Sun, Jiahui Wang, Siran Wang, Zhihong Wang, Chenrui Wei, Shufa Wei, Yonghui Wu, Yuchen Wu, Yihang Xia, Huajian Xin, Fan Yang, Huaiyuan Ying, Hongyi Yuan, Zheng Yuan, Tianyang Zhan, Chi Zhang, Yue Zhang, Ge Zhang, Tianyun Zhao, Jianqiu Zhao, Yichi Zhou, and Thomas Hanwen Zhu. Seed-Prover: Deep and broad reasoning for automated theorem proving. *CoRR*, abs/2507.23726, 2025. doi: 10.48550/ARXIV.2507.23726. URL https://doi.org/10.48550/arXiv.2507.23726.

Andrea Colesanti, Galyna V. Livshyts, and Arnaud Marsiglietti. On the stability of Brunn–Minkowski type inequalities. *Journal of Functional Analysis*, 273(3):1120–1139, 2017. doi: 10.1016/j.jfa.2017.04.008. URL https://arxiv.org/abs/1606.06586. Preprint first posted in 2016, arXiv:1606.06586.

Matthias Cosler, Christopher Hahn, Daniel Mendoza, Frederik Schmitt, and Caroline Trippel. nl2spec: Interactively translating unstructured natural language to temporal logics with large language models. In *Computer Aided Verification (CAV 2023)*, volume 13965 of *Lecture Notes in Computer Science*, pages 383–396, 2023. doi: 10.1007/978-3-031-37703-7_18. URL https://link.springer.com/chapter/10.1007/978-3-031-37703-7_18.

George B. Dantzig. *Linear Programming and Extensions*. Princeton University Press, Princeton, NJ, 1963. URL https://www.rand.org/pubs/reports/R366.html. Hirsch's conjecture, pp. 160 and 168; attributed to W. M. Hirsch, 1957.

Ludwig Danzer. Problem 6. In Werner Fenchel, editor, *Proceedings of the Colloquium on Convexity, Copenhagen, 1965*. Københavns Universitets Matematiske Institut, Copenhagen, 1967. In the collected Problems section.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635, Berlin, Heidelberg, 2021. Springer-Verlag. ISBN 978-3-030-79875-8. doi: 10.1007/978-3-030-79876-5_37. URL https://doi.org/10.1007/978-3-030-79876-5_37.

Aaron Defazio, Francis R. Bach, and Simon Lacoste-Julien. SAGA: A fast incremental gradient method with support for non-strongly convex composite objectives. In Zoubin Ghahramani, Max Welling, Corinna Cortes, Neil D. Lawrence, and Kilian Q. Weinberger, editors, *Advances in Neural Information Processing Systems 27: Annual Conference on Neural Information Processing Systems 2014, December 8-13 2014, Montreal, Quebec, Canada*, pages 1646–1654, 2014. URL https://proceedings.neurips.cc/paper/2014/hash/937964195d6fb3a55cd7cc578165f058-Abstract.html.

Jasper Dekoninck, Nikola Jovanovic, Tim Gehrunger, Kári Rögnvaldsson, Ivo Petrov, Chenhao Sun, and Martin T. Vechev. Beyond benchmarks: MathArena as an evaluation platform for mathematics with LLMs. *CoRR*, abs/2605.00674, 2026. doi: 10.48550/ARXIV.2605.00674. URL https://doi.org/10.48550/arXiv.2605.00674.

Shuo Deng and Kenneth W. Shum. From lecture notes to Lean: Formalizing a textbook on probability theory. *CoRR*, abs/2607.27298, 2026. doi: 10.48550/ARXIV.2607.27298. URL https://doi.org/10.48550/arXiv.2607.27298.

John H. Drew, Charles R. Johnson, and Raphael Loewy. Completely positive matrices associated with M-matrices. *Linear and Multilinear Algebra*, 37(4):303–310, 1994. doi: 10.1080/03081089408818334. URL https://doi.org/10.1080/03081089408818334.

John C. Duchi. Electrical Engineering 364m: The Mathematics of Convexity, 2024. URL https://web.stanford.edu/class/ee364m/.

Moritz Firsching, Paul Lezeau, Salvatore Mercuri, Miklós Z. Horváth, Yaël Dillies, Calle Sönne, Eric Wieser, Fred Zhang, Thomas Hubert, Blaise Agüera y Arcas, and Pushmeet Kohli. Formal Conjectures: An open and evolving benchmark for verified discovery in mathematics. *CoRR*, abs/2605.13171, 2026. doi: 10.48550/ARXIV.2605.13171. URL https://doi.org/10.48550/arXiv.2605.13171.

Richard J. Gardner and Artem Zvavitch. Gaussian Brunn–Minkowski inequalities. *Transactions of the American Mathematical Society*, 362(10):5333–5353, 2010. doi: 10.1090/S0002-9947-2010-04891-3. URL https://doi.org/10.1090/S0002-9947-2010-04891-3.

Nikhil Garg. EconCSLib: AI-assisted Lean formalization for economics & computation research. *CoRR*, abs/2606.13306, 2026. doi: 10.48550/ARXIV.2606.13306. URL https://doi.org/10.48550/arXiv.2606.13306.

I. Ts. Gohberg and A. S. Markus. A problem on covering of convex figures by similar figures. *Izvestiya Moldavskogo Filiala Akademii Nauk SSSR*, 10 (76):87–90, 1960. In Russian.

Yehoram Gordon and Mathieu Meyer. On the volume of unions and intersections of balls in Euclidean space. In Joram Lindenstrauss and Vitali D. Milman, editors, *Geometric Aspects of Functional Analysis*, volume 77 of *Operator Theory: Advances and Applications*, pages 91–101. Birkhäuser, Basel, 1995. doi: 10.1007/978-3-0348-9090-8_9. URL https://doi.org/10.1007/978-3-0348-9090-8_9.

Hugo Hadwiger. Ungelöste Probleme, Nr. 20. *Elemente der Mathematik*, 12:121, 1957. URL https://gdz.sub.uni-goettingen.de/dms/resolveppn/?PPN=GDZPPN002074206.

Harbor Framework Team. Harbor: A framework for evaluating and optimizing agents and models in container environments, 2026. URL https://doi.org/10.5281/zenodo.20953922.

Sidharth Hariharan, Christopher Birkbeck, Seewoo Lee, Ho Kiu Gareth Ma, Bhavik Mehta, Auguste Poiroux, and Maryna Viazovska. Progress in formalizing sphere packing in dimension 8. *arXiv preprint arXiv:2604.23468*, 2026.

J. William Helton and Victor Vinnikov. Linear matrix inequality representation of sets. *Communications on Pure and Applied Mathematics*, 60(5):654–674, 2007. doi: 10.1002/cpa.20155. URL https://arxiv.org/abs/math/0306180. Preprint first posted in 2003, arXiv:math/0306180.

Jean-Baptiste Hiriart-Urruty and Claude Lemaréchal. *Convex Analysis and Minimization Algorithms*, volume 305–306 of *Grundlehren der mathematischen Wissenschaften*. Springer, Berlin, Heidelberg, 1993. Vol. I: Fundamentals, doi: 10.1007/978-3-662-02796-7; Vol. II: Advanced Theory and Bundle Methods, doi: 10.1007/978-3-662-06409-2.

Jean-Baptiste Hiriart-Urruty and Claude Lemaréchal. *Fundamentals of Convex Analysis*. Grundlehren Text Editions. Springer, Berlin, Heidelberg, 2001. ISBN 978-3-642-56468-0. doi: 10.1007/978-3-642-56468-0.

Thomas Hubert, Rishi Mehta, Laurent Sartran, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106):607–613, 2025. doi: 10.1038/s41586-025-09833-y. URL https://doi.org/10.1038/s41586-025-09833-y.

Jiedong Jiang, Wanyi He, Yuefeng Wang, Guoxiong Gao, Yongle Hu, Jingting Wang, Nailin Guan, Peihao Wu, Bryan Dai, Liang Xiao, and Bin Dong. FATE: A formal benchmark series for frontier algebra of multiple difficulty levels. In Carl Vondrick, Bharath Hariharan, Colin Raffel, Lerrel Pinto, Diyi Yang, and Aleksandra Faust, editors, *The Fourteenth International Conference on Learning Representations, ICLR 2026, Rio de Janeiro, Brazil, April 23-27, 2026*. proceedings.iclr.cc, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/11c0318fc7b0315fa9542449b1c9ba32-Abstract-Conference.html.

Gil Kalai. The number of faces of centrally-symmetric polytopes. *Graphs and Combinatorics*, 5:389–391, 1989. doi: 10.1007/BF01788696. URL https://doi.org/10.1007/BF01788696.

Ravi Kannan, László Lovász, and Miklós Simonovits. Isoperimetric problems for convex bodies and a localization lemma. *Discrete & Computational Geometry*, 13:541–559, 1995. doi: 10.1007/BF02574061. URL https://doi.org/10.1007/BF02574061.

W. A. Kirk. A fixed point theorem for mappings which do not increase distances. *The American Mathematical Monthly*, 72(9):1004–1006, 1965. doi: 10.2307/2313345. URL https://doi.org/10.2307/2313345.

Victor Klee. Unsolved problems in intuitive geometry. University of Washington, unpublished problem collection, 1960. URL https://sites.math.washington.edu/~reu/papers/2010/mark/Unsolved_Problems_In_Intuitive_Geometry.pdf. October 1960. Reproduced with comments by Branko Grünbaum in 2010. Problem K4, p. 9.

Victor Klee. Convexity of Chebyshev sets. *Mathematische Annalen*, 142:292–304, 1961. doi: 10.1007/BF01353420. URL https://doi.org/10.1007/BF01353420.

Martin Kneser. Einige Bemerkungen über das Minkowskische Flächenmaß. *Archiv der Mathematik*, 6(5):382–390, 1955. doi: 10.1007/BF01900510. URL https://doi.org/10.1007/BF01900510.

Mario Kummer and Tim Netzer. The generalized Lax conjecture for strictly hyperbolic polynomials, 2026. URL https://arxiv.org/abs/2609.24542.

Peter D. Lax. Differential equations, difference equations and matrix theory. *Communications on Pure and Applied Mathematics*, 11(2):175–194, 1958. doi: 10.1002/cpa.3160110203. URL https://doi.org/10.1002/cpa.3160110203.

Lean FRO. Comparator. https://github.com/leanprover/comparator, 2025. Merged into Lean core as `lake comparator` as of Lean v4.35.0.

Lean FRO. LeanEval: Comparator-based Lean formal mathematics eval. https://github.com/leanprover/lean-eval, 2026.

Chenyi Li, Ziyu Wang, Wanyi He, Yuxuan Wu, Shengyang Xu, and Zaiwen Wen. Formalization of convergence rates of four first-order algorithms for convex optimization. *J. Autom. Reason.*, 69(4), October 2025. ISSN 0168-7433. doi: 10.1007/s10817-025-09741-w. URL https://doi.org/10.1007/s10817-025-09741-w.

Yong Lin, Shange Tang, Bohan Lyu, Jiayun Wu, Hongzhou Lin, Kaiyu Yang, Jia Li, Mengzhou Xia, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-Prover: A frontier model for open-source automated theorem proving. *CoRR*, abs/2502.07640, 2025. doi: 10.48550/ARXIV.2502.07640. URL https://doi.org/10.48550/arXiv.2502.07640.

Yong Lin, Shange Tang, Bohan Lyu, Ziran Yang, Jui-Hui Chung, Haoyu Zhao, Lai Jiang, Yihan Geng, Jiawei Ge, Jingruo Sun, Jiayun Wu, Jiri Gesi, Ximing Lu, David Acuna, Kaiyu Yang, Hongzhou Lin, Yejin Choi, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-Prover-V2: Scaling formal theorem proving with scaffolded data synthesis and self-correction. In Carl Vondrick, Bharath Hariharan, Colin Raffel, Lerrel Pinto, Diyi Yang, and Aleksandra Faust, editors, *The Fourteenth International Conference on Learning Representations, ICLR 2026, Rio de Janeiro, Brazil, April 23-27, 2026*. proceedings.iclr.cc, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/13e8be77982beb73d7ed0bbf122f9f3c-Abstract-Conference.html.

Wentao Long, Yunfei Zhang, Chenyi Li, Li Zhou, Chumin Sun, and Zaiwen Wen. CAM-Bench: A benchmark for computational and applied mathematics in Lean. *CoRR*, abs/2605.17255, 2026. doi: 10.48550/ARXIV.2605.17255. URL https://doi.org/10.48550/arXiv.2605.17255.

László Lovász. On the Shannon capacity of a graph. *IEEE Transactions on Information Theory*, 25 (1):1–7, 1979. doi: 10.1109/TIT.1979.1055985. URL https://doi.org/10.1109/TIT.1979.1055985.

Sirui Lu, Ruixuan Deng, Yanqiao Zhu, and Zhengfeng Ji. Long-horizon autoformalization of a core theorem underlying MIP* = RE. *CoRR*, abs/2609.19814, 2026. doi: 10.48550/ARXIV.2609.19814. URL https://doi.org/10.48550/arXiv.2609.19814.

Kurt Mahler. Ein Übertragungsprinzip für konvexe Körper. *Časopis pro pěstování matematiky a fysiky*, 68(3–4):93–102, 1939a. doi: 10.21136/CPMF.1939.109441. URL https://dml.cz/handle/10338.dmlcz/109441.

Kurt Mahler. Ein Minimalproblem für konvexe Polygone. *Mathematica (Zutphen) B*, 7:118–127, 1939b. URL https://carmamaths.org/resources/mahler/docs/059.pdf.

Theodore Meek, Siyuan Ge, Di Qiu Xiang, Simon Chess, and Vasily Ilin. Formalizing numerical analysis: An agent pipeline and quality audit beyond kernel acceptance. *CoRR*, abs/2606.14000, 2026. doi: 10.48550/ARXIV.2606.14000. URL https://doi.org/10.48550/arXiv.2606.14000.

Daniel Mendoza, Anastasia Mavridou, Andreas Katis, and Caroline Trippel. Automating requirements formalization: Using LLMs and low-complexity distinguishing traces for semantic validation. In *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*, ICSE '26, pages 1314–1326, New York, NY, USA, 2026. Association for Computing Machinery. ISBN 9798400720253. doi: 10.1145/3744916.3787815. URL https://doi.org/10.1145/3744916.3787815.

OpenAI. Codex CLI, 2025a. URL https://github.com/openai/codex. Agentic coding tool.

OpenAI. GPT-5.1 Instant and GPT-5.1 Thinking System Card Addendum. Technical report, OpenAI, November 2025b. URL https://deploymentsafety.openai.com/gpt-5-1.

OpenAI. GPT-5.4 Thinking System Card. Technical report, OpenAI, March 2026a. URL https://deploymentsafety.openai.com/gpt-5-4-thinking.

OpenAI. GPT-5.6 System Card. Technical report, OpenAI, July 2026b. URL https://deploymentsafety.openai.com/gpt-5-6.

OpenAI. Addendum to GPT-6 Astra System Card: GPT-6.1 Sol. Technical report, OpenAI, September 2026c. URL https://deploymentsafety.openai.com/gpt-6-1-sol.

OpenAI. GPT-6 Astra System Card. Technical report, OpenAI, September 2026d. URL https://deploymentsafety.openai.com/gpt-6-astra.

OpenAI. Fixed Points of Nonexpansive Maps in Reflexive Banach Spaces. OpenAI Math Release preprint OAI:Fixed-Points-of-Nonexpansive-Maps-in-Reflexive-Banach-Spaces-September-24-2026, 2026e.

OpenAI. A nine-dimensional counterexample to Borsuk's covering assertion. OpenAI Math Release preprint OAI:A-nine-dimensional-counterexample-to-Borsuks-covering-assertion-September-23-2026, 2026f.

OpenAI. A nonspectrahedral hyperbolicity cone. OpenAI Math Release preprint OAI:A-Nonspectrahedral-Hyperbolicity-Cone-September-24-2026, 2026g.

OpenAI. The logarithmic Brunn–Minkowski conjecture. OpenAI Math Release preprint OAI:The-logarithmic-Brunn-Minkowski-conjecture-September-23-2026, 2026h.

OpenAI. The Mahler Conjecture for General Convex Bodies. OpenAI Math Release preprint OAI:The-Mahler-Conjecture-for-General-Convex-Bodies-September-22-2026, 2026i.

OpenAI. The symmetric Mahler conjecture and its equality cases. OpenAI Math Release preprint OAI:The-symmetric-Mahler-conjecture-and-its-equality-cases-September-22-2026, 2026j.

OpenAI. Petty's projection-volume conjecture in dimensions at least four. OpenAI Math Release preprint OAI:Pettys-projection-volume-conjecture-in-dimensions-at-least-four-September-24-2026, 2026k.

OpenAI. Finite time blowup for Navier–Stokes. https://cdn.openai.com/pdf/32d9f210-8b73-45e0-91bc-82a30aef8a9a/navier-stokes.pdf, 2026l. Lean formalization available at https://github.com/openai/NavierStokesAndEuler.

Clinton M. Petty. Isoperimetric problems. In *Proceedings of the Conference on Convexity and Combinatorial Geometry (Norman, Oklahoma, 1971)*, pages 26–41. University of Oklahoma, Norman, Oklahoma, 1971. URL https://mathscinet.ams.org/mathscinet-getitem?mr=0362057.

Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. *CoRR*, abs/2009.03393, 2020. doi: 10.48550/ARXIV.2009.03393. URL https://doi.org/10.48550/arXiv.2009.03393.

Ebbe Thue Poulsen. Problem 10. *Mathematica Scandinavica*, 2:346, 1954.

Siddhartha Prasad, Skyler Austen, Kathi Fisler, and Shriram Krishnamurthi. Meaningful human-in-the-loop checking of GenAI synthesis for restricted languages. In *40th European Conference on Object-Oriented Programming (ECOOP 2026)*, volume 372 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 22:1–22:31, 2026. doi: 10.4230/LIPIcs.ECOOP.2026.22. URL https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2026.22.

J. Scott Provan and Louis J. Billera. Decompositions of simplicial complexes related to diameters of convex polyhedra. *Mathematics of Operations Research*, 5(4):576–594, 1980. doi: 10.1287/moor.5.4.576. URL https://doi.org/10.1287/moor.5.4.576.

Lushi Pu, Weiming Zhang, Xinheng Xie, Zixuan Fu, Bingxiang He, Hongya Lyu, Xin Li, Jie Zhou, and Yudong Wang. MA-ProofBench: A two-tiered evaluation of LLMs for theorem proving in mathematical analysis. *CoRR*, abs/2606.13782, 2026. doi: 10.48550/ARXIV.2606.13782. URL https://doi.org/10.48550/arXiv.2606.13782.

Karl Reinhardt. Über die dichteste gitterförmige Lagerung kongruenter Bereiche in der Ebene und eine besondere Art konvexer Kurven. *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg*, 10:216–230, 1934. doi: 10.1007/BF02940676. URL https://doi.org/10.1007/BF02940676.

Yuanjie Ren, Jinzheng Li, and Yidi Qi. MerLean: An agentic framework for autoformalization in quantum computation. *CoRR*, abs/2602.16554, 2026. doi: 10.48550/ARXIV.2602.16554. URL https://doi.org/10.48550/arXiv.2602.16554.

Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Z. F. Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *CoRR*, abs/2504.21801, 2025. doi: 10.48550/ARXIV.2504.21801. URL https://doi.org/10.48550/arXiv.2504.21801.

James Renegar. *A mathematical view of interior-point methods in convex optimization*. MPS-SIAM series on optimization. SIAM, 2001. ISBN 978-0-89871-502-6.

R. Tyrrell Rockafellar and Roger J.-B. Wets. *Variational Analysis*, volume 317 of *Grundlehren der mathematischen Wissenschaften*. Springer, Berlin, Heidelberg, 2009. ISBN 978-3-642-02431-3. doi: 10.1007/978-3-642-02431-3. Figures illustrated by Maria Wets.

Mark Schmidt, Nicolas Le Roux, and Francis R. Bach. Minimizing finite sums with the stochastic average gradient. *Math. Program.*, 162(1-2):83–112, 2017. doi: 10.1007/S10107-016-1030-6. URL https://doi.org/10.1007/s10107-016-1030-6.

Omri Solan, Yaar Solomon, and Barak Weiss. On problems of Danzer and Gowers and dynamics on the space of closed subsets of $\mathbb{R}^d$. *International Mathematics Research Notices*, 2017(21): 6584–6598, 2017. doi: 10.1093/imrn/rnw204. URL https://arxiv.org/abs/1510.07179. Preprint first posted in 2015, arXiv:1510.07179.

Zhao Song and Xinzhi Zhang. An $O(1)$ bound for the KLS constant, 2026. URL https://arxiv.org/abs/2610.01447.

P. P. Tammela. An estimate of the critical determinant of a two-dimensional convex symmetric domain. *Izvestiya Vysshikh Uchebnykh Zavedenii. Matematika*, 12(103):103–107, 1970. URL https://mathnet.ru/eng/ivm3798. In Russian.

Tau Ceti Contributors. Tau Ceti. https://github.com/TauCetiProject/TauCeti, 2026.

The mathlib Community. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP 2020, pages 367–381, New Orleans, LA, USA, January 2020. ACM. doi: 10.1145/3372885.3373824. URL https://doi.org/10.1145/3372885.3373824.

Ryan Tibshirani. Modern stochastic methods. Lecture slides for Convex Optimization (10-725), Carnegie Mellon University, 2019. URL https://www.stat.cmu.edu/~ryantibs/convexopt/lectures/modern-sgd.pdf.

George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amitayush Thakur, and Swarat Chaudhuri. PutnamBench: Evaluating neural theorem-provers on the Putnam mathematical competition. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/1582eaf9e0cf349e1e5a6ee453100aa1-Abstract-Datasets_and_Benchmarks_Track.html.

Sumanth Varambally, Thomas Voice, Yanchao Sun, Zhifeng Chen, Rose Yu, and Ke Ye. Hilbert: Recursively building formal proofs with informal reasoning. In Carl Vondrick, Bharath Hariharan, Colin Raffel, Lerrel Pinto, Diyi Yang, and Aleksandra Faust, editors, *The Fourteenth International Conference on Learning Representations, ICLR 2026, Rio de Janeiro, Brazil, April 23-27, 2026*. proceedings.iclr.cc, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/41b95e8ee0e054dab145a7564d9cd58a-Abstract-Conference.html.

Haiming Wang, Mert Unsal, Xiaohan Lin, Mantas Baksys, Junqi Liu, Marco Dos Santos, Flood Sung, Marina Vinyes, Zhenzhe Ying, Zekai Zhu, Jianqiao Lu, Hugues de Saxcé, Bolton Bailey, Chendong Song, Chenjun Xiao, Dehao Zhang, Ebony Zhang, Frederick Pu, Han Zhu, Jiawei Liu, Jonas Bayer, Julien Michel, Longhui Yu, Léo Dreyfus-Schmidt, Lewis Tunstall, Luigi Pagani, Moreira Machado, Pauline Bourigault, Ran Wang, Stanislas Polu, Thibaut Barroyer, Wen-Ding Li, Yazhe Niu, Yann Fleureau, Yangyang Hu, Zhouliang Yu, Zihan Wang, Zhilin Yang, Zhengying Liu, and Jia Li. Kimina-Prover Preview: Towards large formal reasoning models with reinforcement learning. *CoRR*, abs/2504.11354, 2025. doi: 10.48550/ARXIV.2504.11354. URL https://doi.org/10.48550/arXiv.2504.11354.

Qingxiang Wang, Cezary Kaliszyk, and Josef Urban. First experiments with neural translation of informal to formal mathematics. In Florian Rabe, William M. Farmer, Grant O. Passmore, and Abdou Youssef, editors, *Intelligent Computer Mathematics - 11th International Conference, CICM 2018, Hagenberg, Austria, August 13-17, 2018, Proceedings*, volume 11006 of *Lecture Notes in Computer Science*, pages 255–270. Springer, 2018. doi: 10.1007/978-3-319-96812-4_22. URL https://doi.org/10.1007/978-3-319-96812-4_22.

Zichen Wang, Wanli Ma, Zhenyu Ming, Gong Zhang, Kun Yuan, and Zaiwen Wen. M2F: Automated formalization of mathematical literature at scale. *CoRR*, abs/2602.17016, 2026. doi: 10.48550/ARXIV.2602.17016. URL https://doi.org/10.48550/arXiv.2602.17016.

Ke Weng, Lun Du, Sirui Li, Wangyue Lu, Haozhe Sun, Hengyu Liu, and Tiancheng Zhang. Autoformalization in the era of large language models: A survey. *CoRR*, abs/2505.23486, 2025. doi: 10.48550/ARXIV.2505.23486. URL https://doi.org/10.48550/arXiv.2505.23486.

Yuhuai Wu, Albert Qiaochu Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/d0c6bc641a56bebee9d985b937307367-Abstract-Conference.html.

Ran Xin, Chenguang Xi, Jie Yang, Feng Chen, Hang Wu, Xia Xiao, Yifan Sun, Shen Zheng, and Ming Ding. BFS-Prover: Scalable best-first tree search for LLM-based automatic theorem proving. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 32588–32599. Association for Computational Linguistics, 2025. doi: 10.18653/V1/2025.ACL-LONG.1565. URL https://doi.org/10.18653/v1/2025.acl-long.1565.

Zhouliang Yu, Ruotian Peng, Keyi Ding, Yizhe Li, Zhongyuan Peng, Minghao Liu, Yifan Zhang, Zheng Yuan, Huajian Xin, Wenhao Huang, Yandong Wen, Ge Zhang, and Weiyang Liu. FormalMATH: Benchmarking formal mathematical reasoning of large language models. *CoRR*, abs/2505.02735, 2025. doi: 10.48550/ARXIV.2505.02735. URL https://doi.org/10.48550/arXiv.2505.02735.

Yuanhe Zhang, Jason D. Lee, and Fanghui Liu. AI4SLT: Empirical processes in Lean 4 for formal statistical learning theory. In Tong Zhang, Miroslav Dudik, Martin Jaggi, Alekh Agarwal, Sharon Li, Dale Schuurmans, Jerry Zhu, Felix Berkenkamp, Hanze Dong, and Alberto Bietti, editors, *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 156234–156262. PMLR, 06–11 Jul 2026a. URL https://proceedings.mlr.press/v306/zhang26cd.html.

Zhiyuan Zhang, Axel Delaval, Leheng Chen, Jinxuan Chen, Jie Xu, Yuxuan Liao, Jiedong Jiang, Chunlei Liu, and Bin Dong. An AI-assisted formalization of the Poincaré conjecture, 2026b. URL https://arxiv.org/abs/2610.08329.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. MiniF2F: a cross-system benchmark for formal Olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2109.00110.

A Related works, continued

A.1 Formal mathematics benchmarks

Prior works have evaluated the mathematical reasoning abilities of models beyond final answer correctness by additionally verifying the proof. Zheng et al. [2022] introduce MiniF2F, which formalizes high-school and undergraduate exercises as well as olympiad problems. Advanced undergraduate mathematics is more generally covered in FormalMATH [Yu et al., 2025], ProofNet [Azerbayev et al., 2023], and PutnamBench [Tsoukalas et al., 2024]. Benchmarks like FATE [Jiang et al., 2026] for algebra and MA-ProofBench [Pu et al., 2026] for analysis then evaluate models on specific areas of graduate level math.

These benchmarks all evaluate non-agentic proof generation, but recent works have turned to agentic-proving systems. CAM-Bench includes agent evaluations on graduate-level applied mathematics problems [Long et al., 2026]. LeanEval [Lean FRO, 2026], AnnalsChallenge [Browning et al., 2026], and ArXivLean [Dekoninck et al., 2026] also challenge agents to formalize research-level mathematics. Formal Conjectures consists of formalizations of open problems, requiring both novel mathematics as well as a formalization of the proof [Firsching et al., 2026].

A.2 Neural proof generation

Neural proof generation includes tactic-level search [Polu and Sutskever, 2020, Xin et al., 2025, Hubert et al., 2025], whole-proof generation with sampling and refinement [Ren et al., 2025, Wang et al., 2025, Lin et al., 2025, 2026, Chen et al., 2025], and agentic workflows combining informal reasoning with formal verification [Varambally et al., 2026, Achim et al., 2025].

Table 2: Relevant sources for formalized research problems.

Problem	Sources
Bang's affine plank problem	Bang [1951]
Borsuk's partition conjecture ($n \leq 62$)	Borsuk [1933]
Klee's Chebyshev set problem	Klee [1960, Problem K4] (problem collection); Klee [1961, pp. 292–293]
Danzer's problem	Danzer [1967] (original question); Bambah and Woods [1971]; Solan et al. [2017]
Dimensional Brunn–Minkowski conjecture	Gardner and Zvavitch [2010] (Gaussian measure); Colesanti et al. [2017] (log-concave measures)
Generalized Lax conjecture	Lax [1958] (original conjecture); Helton and Vinnikov [2007] (generalized version)
Hadwiger's covering conjecture	Hadwiger [1957] (covering question); Gohberg and Markus [1960] (smaller homothets)
Kalai's 3^d conjecture	Kalai [1989]
Kannan–Lovász–Simonovits conjecture	Kannan et al. [1995] (uniform measures on convex bodies); Bobkov [1999] (all log-concave measures)
Kneser–Poulsen conjecture	Poulsen [1954] and Kneser [1955] (equal radii special case); Gordon and Meyer [1995] (arbitrary radii)
Logarithmic Brunn–Minkowski conjecture	Böröczky et al. [2012]
Mahler's conjecture, general position	Mahler [1939b] (planar special case); Aleksandrov [1967] (general dimension)
Mahler's conjecture, symmetric case	Mahler [1939a]
Order-six Drew–Johnson–Loewy conjecture	Drew et al. [1994]
Petty's projection conjecture	Petty [1971]
Polynomial Hirsch conjecture	Hirsch (1957), reported in Dantzig [1963] (linear bound); Provan and Billera [1980] (polynomial bound)
Fixed points in reflexive Banach spaces	Kirk [1965]
Reinhardt's conjecture	Reinhardt [1934]
Shannon capacity of the 7-cycle	Lovász [1979]
Sharp symmetric slicing conjecture	Ball [1988]

B Natural-language statements of the open problems

This appendix contains the natural-language statement that was used to formalize each research problem. We include relevant sources for each problem in Table 2.

B.1 Bang's affine plank problem

A *plank* in $\mathbb{R}^d$ is a closed slab between two parallel hyperplanes,

$$P = \left\{x \in \mathbb{R}^d : a \leq \langle u, x\rangle \leq b\right\}, \qquad \|u\| = 1,\ a \leq b,$$

and its width is $w(P) = b - a$. Let $K \subseteq \mathbb{R}^d$ be a convex body (compact, convex, with non-empty interior) and let $h_K(u) = \sup_{x\in K}\langle u, x\rangle$ be its support function. The width of K in direction u is

$$w_K(u) = h_K(u) + h_K(-u),$$

the distance between the two supporting hyperplanes of K orthogonal to u, and the *relative width* of a plank P in direction u with respect to K is the ratio $w(P)/w_K(u)$. If K is covered by finitely many planks $P_1, \ldots, P_m$ with directions $u_1, \ldots, u_m$, then their relative widths sum to at least one:

$$K \subseteq \bigcup_{i=1}^{m} P_i \quad \Longrightarrow \quad \sum_{i=1}^{m} \frac{w(P_i)}{w_K(u_i)} \geq 1.$$

B.2 Borsuk's partition conjecture

For $n \le 62$, every bounded set $A \subset \mathbb{R}^n$ with $\operatorname{diam} A > 0$ can be partitioned into $n+1$ subsets of strictly smaller diameter:

$$A = \bigcup_{i=1}^{n+1} A_i, \qquad \operatorname{diam}(A_i) < \operatorname{diam}(A) \text{ for every } i.$$

OpenAI [2026f] disproved the conjecture and provided an accompanying formal Lean proof.

B.3 Klee's Chebyshev set problem

Let H be a real Hilbert space and $C \subseteq H$ nonempty with the property that every point has a *unique* nearest point in C:

$$\forall x \in H,\ \exists!\, c \in C :\ \|x - c\| = \inf_{y \in C} \|x - y\|\,.$$

Must C be convex?

B.4 Danzer's problem

Call $Y \subseteq \mathbb{R}^d$ a *Danzer set* if it meets every convex body of volume one: $Y \cap K \neq \emptyset$ whenever K is convex with $\operatorname{vol}(K) = 1$. For $d \ge 2$, does there exist a Danzer set of **bounded density**?

$$\begin{aligned} \exists\, Y,\ \exists\, C, \quad & (\forall K \text{ convex},\ \operatorname{vol}(K) = 1 \Rightarrow Y \cap K \neq \emptyset) \\ & \wedge\ (\forall x,\ \#\,(Y \cap B(x,1)) \le C)\,. \end{aligned}$$

B.5 The dimensional Brunn–Minkowski conjecture

For every even log-concave μ on $\mathbb{R}^n$, symmetric convex K, L, and $\lambda \in [0,1]$,

$$\mu\,(\lambda K + (1-\lambda)L)^{1/n} \ \ge\ \lambda\,\mu(K)^{1/n} + (1-\lambda)\,\mu(L)^{1/n}.$$

B.6 The generalized Lax conjecture

Let $p \in \mathbb{R}[x_1, \dots, x_n]$ be homogeneous and hyperbolic with respect to e: $p(e) \neq 0$ and $t \mapsto p(x - te)$ has only real roots for every x. Its hyperbolicity cone is

$$\Lambda_+(p, e) = \{x :\ \text{all roots of } t \mapsto p(x - te) \text{ are } \ge 0\}\,.$$

Every such cone is *spectrahedral* — there are symmetric $A_1, \dots, A_n \in \mathcal{S}^m$ with

$$\Lambda_+(p, e) = \left\{ x \in \mathbb{R}^n :\ \sum_i x_i A_i \succeq 0 \right\}.$$

OpenAI [2026g] disproved the conjecture and provided an accompanying formal Lean proof.

B.7 Hadwiger's covering conjecture

Every convex body $K \subset \mathbb{R}^n$ satisfies, for some $x_i \in \mathbb{R}^n$ and $\lambda_i \in (0,1)$,

$$K \ \subseteq\ \bigcup_{i=1}^{2^n} (x_i + \lambda_i K)\,,$$

with 2^n required only for parallelepipeds. Equivalently, 2^n external light sources suffice to illuminate ∂K.

B.8 Kalai's 3^d conjecture

Every centrally symmetric d-polytope $P \subset \mathbb{R}^d$ has at least 3^d nonempty faces:

$$\sum_{k=0}^{d} f_k(P) \;\geq\; 3^d.$$

B.9 The Kannan–Lovász–Simonovits conjecture

There is a universal constant C such that for every n, every log-concave probability measure μ on $\mathbb{R}^n$, and every Borel $A \subseteq \mathbb{R}^n$,

$$\mu^+(A) \;\geq\; \frac{1}{C\,\|\Sigma_\mu\|_{\mathrm{op}}^{1/2}}\,\min\{\mu(A),\,1-\mu(A)\}\,,$$

where Σ_μ is the covariance and $\mu^+(A) \;=\; \liminf_{\varepsilon\to 0^+}\varepsilon^{-1}\,(\mu(A_\varepsilon)-\mu(A))$ is the Minkowski boundary measure.

Song and Zhang [2026] proved the conjecture.

B.10 Kneser–Poulsen conjecture

Let $n, m \geq 1$ and let $p_1, \ldots, p_m$ and $q_1, \ldots, q_m$ be two configurations of points in $\mathbb{R}^n$ such that the second is a *contraction* of the first: every pairwise distance weakly decreases,

$$\|p_i - p_j\| \geq \|q_i - q_j\| \quad \text{for all } i, j.$$

Fix radii $r_1, \ldots, r_m > 0$, the same radii for both configurations, and write $B(x, r)$ for the ball of radius r centred at x. Then contracting the centres cannot increase the volume of the union of the balls:

$$\mathrm{vol}\left(\bigcup_{i=1}^{m} B(p_i, r_i)\right) \geq \mathrm{vol}\left(\bigcup_{i=1}^{m} B(q_i, r_i)\right).$$

B.11 The logarithmic Brunn–Minkowski conjecture

For symmetric convex bodies $K, L \subset \mathbb{R}^n$ and $\lambda \in [0, 1]$, set

$$(1-\lambda)\cdot K +_0 \lambda\cdot L \;:=\; \left\{x :\; \langle x, u\rangle \leq h_K(u)^{1-\lambda} h_L(u)^{\lambda}\ \forall u \in \mathbb{S}^{n-1}\right\}.$$

Then

$$\mathrm{vol}\left((1-\lambda)\cdot K +_0 \lambda\cdot L\right) \;\geq\; \mathrm{vol}(K)^{1-\lambda}\,\mathrm{vol}(L)^{\lambda}.$$

OpenAI [2026h] proved the conjecture and provided an accompanying formal Lean proof.

B.12 Mahler's conjecture, general position

For every convex body $K \subset \mathbb{R}^n$,

$$\min_{z\in \operatorname{int} K} \mathrm{vol}(K)\cdot \mathrm{vol}\left((K-z)^\circ\right) \;\geq\; \frac{(n+1)^{n+1}}{(n!)^2},$$

with equality exactly for simplices.

OpenAI [2026i] proved the conjecture and provided an accompanying formal Lean proof.

B.13 Mahler's conjecture, symmetric case

For every convex body $K \subset \mathbb{R}^n$ with $K = -K$,

$$\operatorname{vol}(K) \cdot \operatorname{vol}(K^\circ) \;\geq\; \frac{4^n}{n!},$$

with equality for the cube, the cross-polytope, and every Hanner polytope.

OpenAI [2026j] proved the conjecture and provided an accompanying formal Lean proof.

B.14 The order-six Drew–Johnson–Loewy conjecture

A matrix $A \in \mathbb{R}^{6\times 6}$ is *completely positive* if it has a nonnegative Gram factorisation:

$$\mathrm{CP}(A) \;:\Longleftrightarrow\; \exists\, r \in \mathbb{N},\ \exists\, B \in \mathbb{R}^{6\times r}, \quad B_{ij} \geq 0 \quad \text{and} \quad A = BB^{\mathsf{T}}.$$

Every completely positive 6×6 matrix has a nonnegative Gram representation with at most nine vectors:

$$\forall\, A \in \mathbb{R}^{6\times 6}, \quad \mathrm{CP}(A) \implies \exists\, C \in \mathbb{R}^{6\times 9}_{\geq 0}, \quad A = CC^{\mathsf{T}}.$$

Allowing exactly nine columns is equivalent to allowing at most nine, since unused columns may be zero.

B.15 Petty's projection conjecture

The *projection body* ΠK of a convex body $K \subset \mathbb{R}^n$ is the convex body whose support function is the shadow area of K:

$$h_{\Pi K}(u) \;=\; \operatorname{vol}_{n-1}\left(K \,|\, u^{\perp}\right), \qquad u \in \mathbb{S}^{n-1},$$

where $K \,|\, u^{\perp}$ is the orthogonal projection of K onto the hyperplane $u^{\perp}$. Petty conjectured that for $n \geq 3$ the affine-invariant functional

$$F(K) \;=\; \operatorname{vol}(\Pi K) \cdot \operatorname{vol}(K)^{1-n}$$

is minimised over all convex bodies if and only if K is an ellipsoid.

OpenAI [2026k] proved the conjecture for $n \geq 4$ and provided an accompanying formal Lean proof.

B.16 The polynomial Hirsch conjecture

There is a polynomial q such that every d-dimensional polytope P with n facets has vertex-edge graph $G(P)$ of combinatorial diameter

$$\operatorname{diam}\, G(P) \;\leq\; q(n, d).$$

B.17 The fixed point property for reflexive Banach spaces

Let X be a reflexive Banach space, $C \subseteq X$ nonempty, closed, bounded and convex, and $T : C \to C$ nonexpansive, i.e. $\|Tx - Ty\| \leq \|x - y\|$ for all $x, y \in C$. Must T have a fixed point?

OpenAI [2026e] proved the conjecture and provided an accompanying formal Lean proof.

B.18 Reinhardt's conjecture

Let $K \subseteq \mathbb{R}^2$ be a convex disc (compact, convex, with non-empty interior). A *packing* of the plane by congruent copies of K is a sequence $(g_i)_{i\in\mathbb{N}}$ of isometries of $\mathbb{R}^2$ such that the sets $g_i(K)$ have pairwise disjoint interiors. Its density is the asymptotic fraction of area covered,

$$\delta\left(K, (g_i)\right) = \limsup_{r\to\infty} \frac{\operatorname{vol}\left(\bigcup_i g_i(K) \cap B(0,r)\right)}{\operatorname{vol}\left(B(0,r)\right)},$$

and the packing density of K is the supremum over all packings,

$$\delta(K) = \sup_{(g_i)} \delta\left(K, (g_i)\right).$$

Call K centrally symmetric if $x \in K$ exactly when $-x \in K$. Then the packing density of every centrally symmetric convex disc is at least that of the smoothed octagon:

$$\delta(K) \geq \frac{8 - 4\sqrt{2} - \log 2}{2\sqrt{2} - 1} \approx 0.902414.$$

B.19 Shannon capacity of the 7-cycle

For finite simple graphs G and H, the strong product $G \boxtimes H$ has vertex set $V(G) \times V(H)$, with (a_1, b_1) adjacent to (a_2, b_2) if and only if the two vertices are distinct and, in each coordinate separately, the entries are either equal or adjacent. Write $G^{\boxtimes k}$ for the k-fold strong power, and $\alpha(G)$ for the independence number, the largest number of pairwise non-adjacent vertices. The Shannon capacity of G is

$$\Theta(G) = \sup_{k \geq 1} \alpha\left(G^{\boxtimes k}\right)^{1/k}.$$

Operationally, $\Theta(G)$ is the effective alphabet size per symbol achievable for zero-error transmission over a channel whose confusable symbol pairs are the edges of G. Let C_7 denote the cycle on 7 vertices. Then

$$\Theta(C_7) = \frac{7\cos(\pi/7)}{1 + \cos(\pi/7)}.$$

B.20 Sharp symmetric slicing conjecture: is the cube the maximiser of the isotropic constant?

For a convex body $K \subset \mathbb{R}^n$ of volume 1 with barycentre at the origin, let $L_K = \det\left(\int_K x x^{\top}\,\mathrm{d}x\right)^{1/(2n)}$. For centrally symmetric K,

$$L_K \leq L_{Q^n} = 1/\sqrt{12} \qquad \text{with} \qquad Q^n = \left[-\frac{1}{2}, \frac{1}{2}\right]^n.$$

C Textbook results, continued

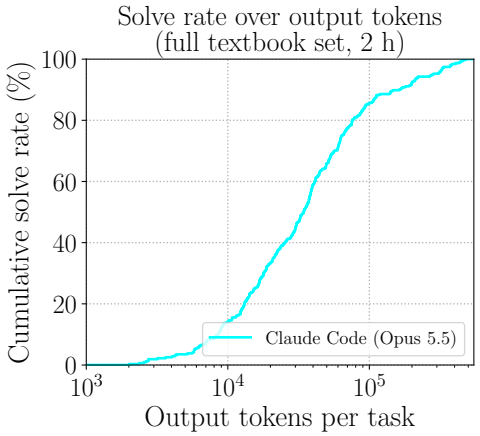
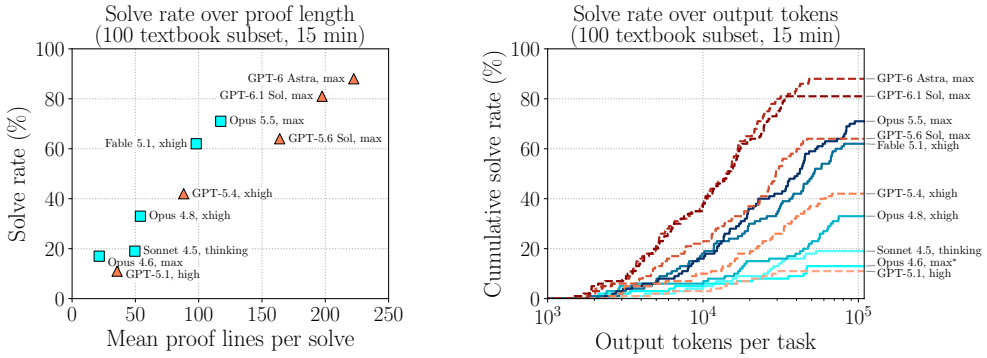


Figure 8: The cumulative solve rate over total outputs tokens for Claude Code with Opus 5.5 on full set of textbook results with a two hour timeout.



(a) (b)

Figure 9: Textbook statements with longer formal proofs are challenging for agents. GPT-6 Astra's 88 successful proofs averaged 222.5 lines, while GPT-5.1's 11 successful proofs averaged 35.5 lines (Figure 9a). Current frontier agents can solve more tasks while using fewer output tokens (Figure 9b). *For Opus 4.6, four of its seventeen successful proofs are omitted because Claude Code 2.1.42 does not record correct output token counts for runs stopped at the evaluation time limit.

D Open problems results, continued

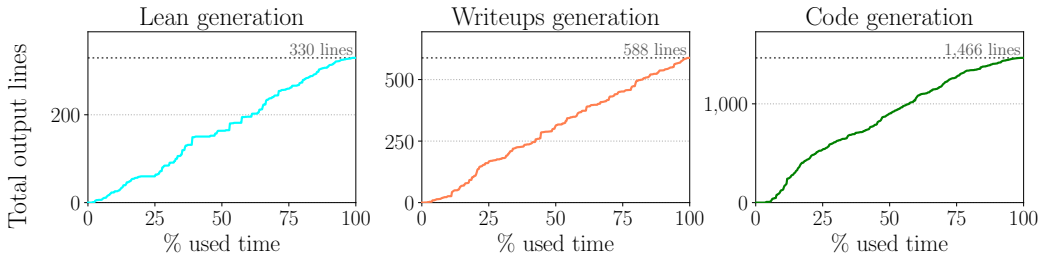


Figure 10: The figures show, for the $100 budget open problems evaluation, the total number of lines of Lean, natural language writing, and other code generated over the course of each trial, averaged over the 20 trials. Lean code and writeup generation occurs throughout the course of the trial, while code generation decreases in the latter half of the trial.

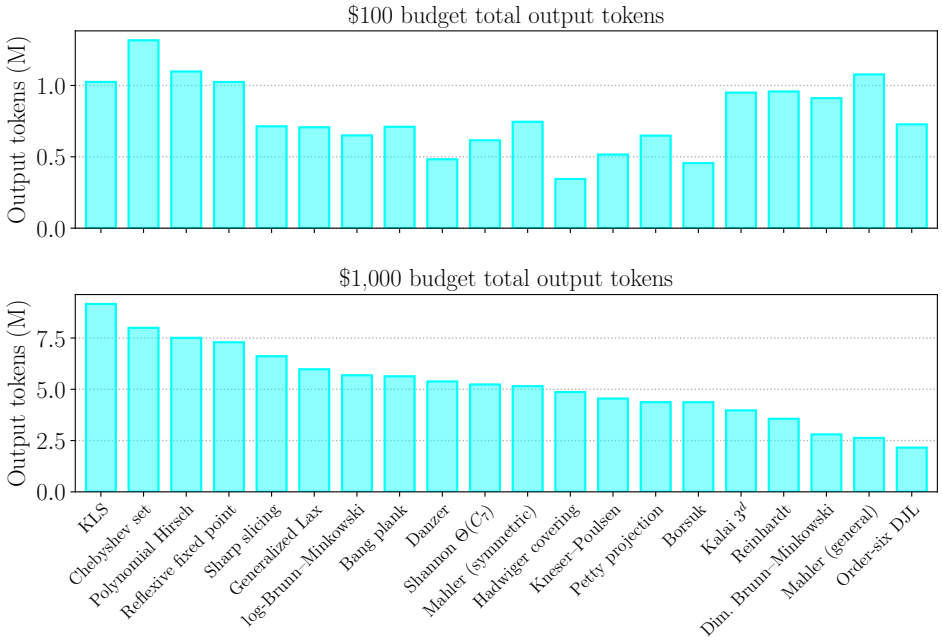


Figure 11: The total number of output tokens produced by Claude Code with Fable 5.1 on each open problem. For the $100 budget runs, agents generated between 0.3 million and 1.3 million tokens, while agents produced between 2.1 million and 9.2 million tokens in the $1,000 runs.

E Frontier models and agent harness versions

Table 3 lists each frontier model and the version of the agent harness we evaluated on the 100 textbook tasks.

Table 3: Frontier models, agent harnesses, and harness versions used for the textbook tasks.

Model	Harness	Version
Claude Sonnet 4.5 [Anthropic, 2025b]	Claude Code	2.0.0
Claude Opus 4.6 [Anthropic, 2026c]	Claude Code	2.1.42
Claude Opus 4.8 [Anthropic, 2026d]	Claude Code	2.1.154
Claude Fable 5.1 [Anthropic, 2026b]	Claude Code	2.1.257
Claude Opus 5.5 [Anthropic, 2026e]	Claude Code	2.1.285
GPT-5.1 [OpenAI, 2025b]	Codex	0.58.0
GPT-5.4 [OpenAI, 2026a]	Codex	0.112.0
GPT-5.6 Sol [OpenAI, 2026b]	Codex	0.144.1
GPT-6 Astra [OpenAI, 2026d]	Codex	0.153.1
GPT-6.1 Sol [OpenAI, 2026c]	Codex	0.159.1

F Autoformalization setup, task instructions, and prompts.

F.1 Task setup

For each textbook autoformalization example, the agent works in an environment containing Lean v4.28 with Mathlib as well as PDF copies of all sources referenced for the formalization project. At a high level, the agent is asked to formalize two statements from different textbooks sources and all definitions necessary to state them in a project-wide definitions file `LeanWorkspace/Defs.lean`.

The evaluation occurs in two stages. In the first stage, the agent formalizes a result from *Fundamentals of Convex Analysis* in a project whose `Defs.lean` file is empty, it contains only `import Mathlib`. In the second stage, a new session of the same agent receives all authored Lean files from the first stage and formalizes a result from another textbook. The agent may edit the existing files.

Agents are given a time limit of 30 minutes for each stage. We ran Codex with GPT-6 Astra, Claude Code with Fable 5.1, and Claude Code with Opus 5.5 with two prompts, a basic prompt and a detailed prompt, given in Section F.2. The detailed prompt includes a guide containing specific instructions about formalization choices that are based on the actual human-authored formalizations.

Tasks. The pairs of textbook results are listed in Table 4.

Table 4: The pairs of textbook results.

First stage	Result (PDF page)	Second stage	Result (PDF page)
`FCA_chap_B_4_1_4`	*Fundamentals of Convex Analysis*, Chapter B, Theorem 4.1.4 (p. 122)	`BC_chap_20_25`	*Convex Analysis and Monotone Operator Theory in Hilbert Spaces*, Theorem 20.25 (p. 361)
`FCA_chap_B_1_2_1`	*Fundamentals of Convex Analysis*, Chapter B, Proposition 1.2.1 (p. 87)	`BC_chap_9_20`	*Convex Analysis and Monotone Operator Theory in Hilbert Spaces*, Theorem 9.20 (p. 175)
`FCA_chap_B_3_2_4_ii`	*Fundamentals of Convex Analysis*, Chapter B, Proposition 3.2.4, second part (p. 117)	`RW_chap_3_26`	*Variational Analysis*, Theorem 3.26 (p. 102)

Sources. The following PDFs were included in the environment.

- *Fundamentals of Convex Analysis* and *Convex Analysis and Minimization Algorithms I*;

- *Variational Analysis*;
- *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*;
- *Convex Optimization: Algorithms and Complexity*;
- *A mathematical view of interior-point methods in convex optimization*, Chapter 2;
- Ten exercise sets of EE364M, Duchi 2024;
- Schmidt et al. [2017], Defazio et al. [2014], and Tibshirani's lecture slides *Modern Stochastic Methods* for Convex Optimization 10-725 at Carnegie Mellon University [Tibshirani, 2019].

F.2 Task instruction template

The two stages share one instruction template and differ only in the header. The second-stage header adds a paragraph stating that the project already contains other formalizations, which the agent may edit. Each header is followed by the shared information below. The detailed prompt adds the sentence "Follow the formalization guide below." to the last paragraph of each header and ends with `{guide}`, which is replaced by the formalization guide in Section F.2.1. Each task fills in `{result_title}`, `{source_short}`, `{source_citation}`, `{pdf_page}`, `{source_file}`, `{module}`, `{theorem_name}` and `{time_budget}` (30 minutes).

First-stage header.

```
# Formalize {result_title} ({source_short})

Write a Lean 4 statement of **{result_title}** from {source_citation} - PDF page {pdf_page}
     of `/app/corpus/{source_file}`.

Put it in `/app/autoformalize/LeanWorkspace/{module}.lean`, the file should have a theorem
     named `{theorem_name}` whose proof is `sorry`. Do not prove it. The project keeps its
     definitions in `/app/autoformalize/LeanWorkspace/Defs.lean`.

Formalize this at a suitable level of generality. Your statement will become part of a
     project that formalizes results from every source in the corpus.
```

Second-stage header.

```
# Formalize {result_title} ({source_short})

Write a Lean 4 statement of **{result_title}** from {source_citation} - PDF page {pdf_page}
     of `/app/corpus/{source_file}`.

Put it in `/app/autoformalize/LeanWorkspace/{module}.lean`, the file should have a theorem
     named `{theorem_name}` whose proof is `sorry`. Do not prove it. The project keeps its
     definitions in `/app/autoformalize/LeanWorkspace/Defs.lean`.

Formalize this at a suitable level of generality. Your statement will become part of a
     project that formalizes results from every source in the corpus.

The project already contains formalizations of other results in the corpus. You may edit
     them, but the package must still build.
```

Shared information.

```
**Time.** {time_budget}. Whatever is on disk at the end is your submission.

## Environment

- `/app/corpus/` holds the books and papers as PDFs (titles in `/app/corpus/INDEX.md`).
- The Lake package is `/app/autoformalize/` (Lean v4.28.0, Mathlib prebuilt). Check your
     file with `lake env lean LeanWorkspace/{module}.lean` or `lake build LeanWorkspace`
     from there.
- Web access is permitted.
- `/app/scratch/` is for notes and extracted text; it is not assessed.
```

```
## Format (checked by `submit` and the grader)

We will check that `lake build LeanWorkspace` succeeds and the file has a single theorem
    `{theorem_name}` whose proof is `sorry`.
```

F.2.1 Formalization guide

```
## Formalization guide

The steps below are the ones to follow when formalizing a result from one of the sources
    into the corpus. Each step gives the decision to make, the Lean pattern to write, and
    what to check.

### Step 1 - Read the source completely before writing anything

Find the result and read, in order: the result itself; the definitions it uses, where the
    source defines them (not from memory); and the chapter's standing assumptions, which
    the statement of the result usually leaves out. Standing assumptions are easy to miss
    and they change the statement. A chapter may fix a norm, a space or a class of
    functions once at its start. Hiriart-Urruty & Lemaréchal first define finite-valued
    convex functions on a convex set and then work mostly with functions into ℝ ∪ {+∞}, so
    check which one a result uses. Rockafellar-Wets' "proper" means never −∞ and not
    identically +∞. Bauschke-Combettes' `Γ₀(H)` bundles proper, lower semicontinuous and
    convex. Duchi's exercises may fix a probability measure for a whole sheet.

In a comment at the top of the file, write down the hypotheses the source states, including
    the standing ones, and the exact conclusion, with the source's indexing and constants.
    Everything you write afterwards is checked against this list.

### Step 2 - Choose the ambient setting

Choose the type of the space first; everything else depends on it.

- If the statement mentions coordinates, the standard basis, a matrix, or the dimension `n`
    as a quantity (a rate with `n` in it, a count of vectors), use `EuclideanSpace ℝ (Fin
    n)`.
- Otherwise, for Hiriart-Urruty & Lemaréchal (both books), Rockafellar-Wets and Renegar,
    which work in ℝ^n, use an abstract finite-dimensional real inner-product space, `{E :
    Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]`, with a
    second space `F` of the same kind when the result involves a map between two spaces.
    Every such space is isometric to some ℝ^n, so nothing is lost and the statement is not
    tied to coordinates.
- For Bauschke-Combettes, use a real Hilbert space: `[CompleteSpace E]` instead of
    `[FiniteDimensional ℝ E]`. Do not add finite dimension; the book's results are
    infinite-dimensional.
- For Bubeck and the SAG/SAGA sources, use an inner-product space as above in the Euclidean
    chapters, and `EuclideanSpace ℝ (Fin n)` where the dimension enters the rate or the
    algorithm acts on coordinates. Where a chapter works with a norm that need not come
    from an inner product, make that norm the space's own norm (`[NormedAddCommGroup E]
    [NormedSpace ℝ E]`, no inner product) and build everything else from the structure the
    source actually uses.
- For Duchi's exercises, use `EuclideanSpace ℝ (Fin n)` with Lebesgue measure, or the
    probability measure the exercise names.

Check that the setting supplies exactly the structure the source uses. Structure the source
    never uses (an inner product, coordinates, completeness, finite dimension) should not
    appear in the statement, and structure it does use must.

Instances: Mathlib's `gradient`, and everything built on it (Hessians, local norms), goes
    through the Riesz isomorphism and needs `[CompleteSpace E]`. A definition over an
    inner-product space that mentions `gradient` must take `[CompleteSpace E]` as an
    instance argument; a theorem that assumes `[FiniteDimensional ℝ E]` gets completeness
    automatically. Measurability of random iterates needs `[MeasurableSpace E] [BorelSpace
    E]`.

### Step 3 - Choose the value type of each function
```

If a function may take the value +∞ in the source's setting (indicator functions, conjugates, support functions, anything "proper" or "closed"), it is `f : E → EReal`, and:

- `effDom f := {x | f x < ⊤}`, and proper is `(∀ x, ⊥ < f x) ∧ (effDom f).Nonempty`.
- Convexity is convexity of the epigraph: `epigraph f := {p : E × ℝ | f p.1 ≤ p.2}` and `ConvexEReal f := Convex ℝ (epigraph f)`. This is correct for every extended-real function. The inequality `f (t • x + (1 - t) • y) ≤ t * f x + (1 - t) * f y` in `EReal` agrees with it only when `f` never takes `⊥`, so do not use the inequality as the definition. A hypothesis `∀ x, ⊥ < f x`, or properness, belongs in a statement only when the source assumes it, not as a way to make convexity behave.
- Lower semicontinuity is Mathlib's `LowerSemicontinuous`.
- A supremum or infimum is `sSup` or `sInf` of an image set, e.g. the conjugate `fun v => sSup ((fun x => ((inner ℝ v x : ℝ) : EReal) - f x) '' Set.univ)` and the support function `fun w => sSup ((fun v => ((inner ℝ v w : ℝ) : EReal)) '' A)`.
- The asymptotic (recession, horizon) function is defined through the epigraph: `asymptoticFun f := funFromSet (horizonCone (epigraph f))`, that is `epi f'_∞ = (epi f)^∞`, with the sequential horizon cone `horizonCone C := {x | ∃ xν ∈ C, λν ↘ 0, λν • xν → x}`. It has no base point. Difference-quotient formulas such as `sup_{t>0} (f (x₀ + t • d) − f x₀) / t` are theorems about it, not its definition.
- The proximal mapping is set-valued: `proximalMapping f lam x := {w | ∀ (z : E), f w + (2 * lam)⁻¹ * ‖w - x‖ ^ 2 ≤ f z + (2 * lam)⁻¹ * ‖z - x‖ ^ 2}`, and `moreauEnvelope f lam x` is the corresponding infimum. An algorithm's proximal step is this mapping applied to the gradient step.
- A real-valued function is passed to an `EReal` definition through `liftRealtoEReal f := fun x => (f x : EReal)`.

If the function is real-valued in the source (algorithms, smooth objectives), keep it `E → ℝ` and use Mathlib's `ConvexOn ℝ s f`, `StrictConvexOn ℝ s f`, `StrongConvexOn s m f`, `LipschitzWith (Real.toNNReal L) f` and `LipschitzOnWith`. `StrongConvexOn` uses the modulus convention `m/2 · a·b·‖x − y‖²`, so a source's "strongly convex with modulus c" in the `c/2` convention is `StrongConvexOn s c f`.

Never use `WithTop ℝ`, and never encode +∞ by a side condition on a real-valued function.

Step 4 - For each concept, reuse, generalize, or define afresh

Aim for one definition per concept, shared by every source that uses it, and as few new definitions as possible. This is a judgement about what the concept is, not a check that two formulas match. For each concept in the statement, go through the following in order.

0. The project. Before anything else, read what the project already has: its shared definitions file and the existing results. A definition already there is the first candidate, and the comparison in item 2 applies to it as to any source's. Then see "Working in the existing project" below for what to do when the project's definition, or an existing result, does not fit.
1. Mathlib. If Mathlib has the concept, use it, even when its defining formula differs from the source's. The exceptions are a Mathlib definition that is a different concept under a similar name or shape, and one that does not cover the source's objects.
2. The other sources in the corpus. Find every source that defines the concept and compare what each definition covers, not how it is written.
   - If one source's definition is a special case of another's, use the general one; the special case is an instance of it.
   - If one definition covers a larger class and the sources prove that their formulas agree with it, use the larger one, even when a book presents the narrower formula first for teaching. Each book's formula is then a theorem of that book.
   - Keep two definitions separate only when neither is more general: they cover the same objects, and their agreement is a substantive theorem. Merging them would hide that theorem inside a definition.
3. State the result with the definitions chosen above, not with the source's own formula when that formula is a special case. The source's formula appears in the statement only when the result is about it, for instance when the result is the book's equivalence theorem. Likewise, when the source defines a method as an instance of a more general method in the corpus, state it as that instance.
4. Only then define something new. Make a new definition as general as the sources that use the concept require, and no more: do not generalize the ambient space, the value type or the quantification beyond what some source in the corpus uses.

Patterns for common kinds of concept:

- A multifunction is `T : E → Set E`. Its properties are stated on its graph, quantifying over `v₀ ∈ T x₀` and `v₁ ∈ T x₁`; its domain is `{x | (T x).Nonempty}`; restricting it to a set means making its values `∅` outside the set, not adding a set argument.
- A property of a function relative to a set `𝒳` (the source's function lives on `𝒳`) quantifies over `𝒳` only, `∀ y ∈ 𝒳, ...`, not over the whole space.
- A set defined by an inequality (cone, polar cone, normal cone, face, sublevel set) is written as that set, e.g. `{s | ∀ y ∈ C, ⟪s, y - x⟫ ≤ 0}`.
- A projection is the set `{w ∈ C | ∀ z ∈ C, ‖w - x‖ ≤ ‖z - x‖}`, or the chosen point `pC x C hC₁ hC₂ hC₃` when closedness, convexity and nonemptiness are available as hypotheses.
- An algorithm is a predicate `Is<Method>Sequence` on its iterates `x : ℕ → E` (or `ℕ → Ω → E` when random) that lists the initialization and the step. Write an explicit step as an equation, `x (t + 1) = x t - η • gradient f (x t)`. Write an argmin step (a projection or a Bregman step) as `IsMinOn (fun z => ...) 𝒳 (x (t + 1))`; this restricts nothing when the source proves the minimizer exists. A proximal step uses `proximalMapping` (Step 3). Step sizes and constants are parameters, and the values the source prescribes are passed where the theorem uses the predicate.

Working in the existing project

The project is one formalization, not a set of files that happen to compile together. Definitions live in the project's shared definitions file; each result holds only its theorem. When your result shares a concept with results already in the project, the project should end with one definition of that concept, used by all of them.

- If the project's definition is a special case of the one your source needs, generalize the existing definition rather than adding a second one beside it, and restate the existing results that use it through the instance. A second definition of the same concept under a different name is the outcome to avoid.
- If the project's definition covers fewer objects than the larger-class definition of Step 4, item 2, switch to the larger one and restate the existing results; the narrower formula becomes a theorem, not a definition.
- If an existing result writes a concept out inline, as a formula in its hypotheses or conclusion, and your result needs the same concept, give it a definition in the shared definitions file and rewrite the existing result to use it.
- Keep each existing result faithful to its own source while you change it: the same hypotheses and conclusion, checked as in Step 5, only expressed through the shared definitions.
- Keep definitions separate only for the reasons in Step 4, item 2, never because changing an existing file is more work.

Step 5 - Write the theorem

Order the binders: the spaces and their instances; the data (sets, functions, iterates, oracle outputs); the constants; the hypotheses, in groups with a one-line comment each (constraint set, function, algorithm, oracle); then the conclusion. Name each hypothesis after its object and property: `(hf_convex : ConvexOn ℝ 𝒳 f)`, `(h𝒳_compact : IsCompact 𝒳)`, `(hx_star : IsMinOn f 𝒳 x_star)`. Props are `UpperCamelCase`, usually `Is...`; data are `lowerCamelCase`.

Fidelity rules, each checked against the list from Step 1:

- Keep every hypothesis the source states, in the source's form, even when it is implied by another hypothesis or by a definition already assumed. Do not drop any, and do not add hypotheses the source does not have: no extra continuity, differentiability, finiteness or compactness.
- Keep the form of the conclusion. A quantity the source defines by an equation stays an equation, not a bound; a named minimizer `x*` stays a parameter with `IsMinOn f 𝒳 x_star` and `x_star ∈ 𝒳`; a supremum is `sSup` of the image set; an expectation is `∫ ω , ... ∂μ`.
- Sequences are indexed from `0`. The source's `x₁` is `x 0`, the sum over `x₁ ... x_t` is `∑ s ∈ Finset.range t, x s`, and schedules shift with the index: a step `2/(s+1)` for `s ≥ 1` becomes `fun s => 2 / (s + 2)` for `s ≥ 0`, and a bound `c/(t+1)` for `t ≥ 1` becomes `c/(t+2)` for `t ≥ 0`. Check every constant after the shift.
- Make casts explicit, `(t : ℝ)`, `(m : ℝ)`, and avoid natural-number subtraction in real expressions.

Step 6 - Add the bookkeeping Lean needs, and audit for junk values

Lean makes partial operations total, so a statement can be true, or a hypothesis vacuous, for reasons the source never considered. Add the hypotheses that rule this out; they are part of what the source means, not additions to it. Go through the statement operation by operation:

- `sSup S` and `sInf S` are `0` when `S` is empty or unbounded. Either the hypotheses make `S` nonempty and bounded, or you are transcribing the source's own definition and a degenerate case only makes the hypotheses inconsistent; know which.
- The Bochner integral `∫ ω, g ω ∂μ` is `0` when `g` is not integrable, so a hypothesis `∫ ... ≤ B` is vacuous without `Integrable`. Pair every expectation bound with its integrability hypothesis, or state the bound with the lower integral `∫⁻`. Every integrand in a conclusion needs an (ae-)measurability hypothesis unless it follows from the others; a function is not measurable just because it is convex on a set.
- `a / b` is `0` when `b = 0`. A step size or rate that divides by a constant needs that constant positive, and one that divides by `t − 1` needs `2 ≤ t`.
- `gradient f x` and `fderiv ℝ f x` are `0` where `f` is not differentiable, so every use needs a differentiability hypothesis at that point.
- `Real.sqrt` and `Real.log` are `0` outside their domains; check that their arguments are nonnegative or positive under the hypotheses.
- Random objects: iterates satisfy `∀ t, Measurable (x t)`; conditional expectations are taken only of integrable functions (`condExp` of a non-integrable function is `0`); independence and sampling laws are stated explicitly.

Step 7 - Encode stochastic statements in the standard way

Work on a probability space `{Ω : Type*} [MeasurableSpace Ω] (μ : Measure Ω) [IsProbabilityMeasure μ]`. Iterates are `x : ℕ → Ω → E`, with `∀ t, Measurable (x t)` inside the sequence predicate. Random indices are `I : ℕ → Ω → Fin m` with `ProbabilityTheory.iIndepFun I μ`, the uniform law `∀ t i, μ (I t ⁻¹' {i}) = (m : ENNReal)⁻¹`, and independence of each sample from the current state, `ProbabilityTheory.IndepFun (I t) (fun ω => (x t ω, ...)) μ`.

A conditional expectation given a random variable `y` is `μ[g | MeasurableSpace.comap y inferInstance]`. Condition on exactly what the source conditions on, neither a smaller nor a larger σ-algebra. A condition on a conditional expectation holds `∀ᵐ ω ∂μ`, since conditional expectations are defined only up to null sets, while the algorithm's own steps hold for every `ω`, as in the source. Expectations are Bochner integrals with their integrability hypotheses (Step 6).

Step 8 - Compile and check

Your file must compile with no errors and exactly one `sorry` warning, at the theorem, and the whole package must build, so every file you changed still compiles. Then check that the theorem has the required name; every definition and auxiliary lemma is complete; there is no `axiom` or `opaque`; the placeholder statement is gone; the pinned package files are untouched; and the hypotheses and conclusion match the list from Step 1 item by item.

Mistakes to avoid

- Transcribing the source's notation instead of its concepts: use the structure the source's definitions actually rely on, not the one its formulas happen to be written in.
- Writing a new definition for a concept Mathlib or the corpus already has, because its formula differs from the source's.
- Adding a general definition next to the project's narrower one instead of generalizing it, or leaving an existing result's inline formula in place when your result defines the same concept.
- Using a book's first-presented formula as the definition when the corpus has a definition of the same concept that covers more.
- Dropping a hypothesis because it is derivable, or adding one because it makes the proof easier.
- Forgetting the index shift, or shifting the iterate but not the step-size schedule or the constant.
- Expectation bounds without integrability; conclusions that integrate a function that need not be measurable; `sSup` over a set that can be empty.

```
- Restricting a result to a smaller setting than the source's (finite dimension where the
    source has a Hilbert space, coordinates where it has none).
```

G Autoformalization examples

In this section we highlight the key differences between the formalizations generated by AI agents compared to human authored formalizations. Each Lean snippet is labeled with the file it comes from. We show only the code needed for the comparison.

G.1 *Fundamentals of Convex Analysis*, Chapter B, Theorem 4.1.4 → *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*, Theorem 20.25 (`FCA_chap_B_4_1_4` → `BC_chap_20_25`)

Under the basic prompt, none of the agents revised the formalization of Theorem 4.1.4 to use set-valued monotonicity after formalizing Theorem 20.25. GPT-6 Astra and Opus 5.5 initially inlined the single-valued definition for Theorem 4.1.4, then separately defined set-valued monotonicity for Theorem 20.25 without updating the earlier formalization. Fable 5.1 placed both definitions in the shared definition file. Although, it explicitly noted there exists a set-valued generalization from Bauschke and Combettes, it still chose to use the single-valued definition for Theorem 4.1.4.

Under the detailed prompt, all three agents defined and used the set-valued definition of monotonicity for both theorems.

G.1.1 Basic prompt

GPT-6 Astra, Codex, attempt 1

First stage (`FCA_chap_B_4_1_4`).

`LeanWorkspace/Defs.lean` contains only `import Mathlib`.

`LeanWorkspace/FCA_chap_B_4_1_4.lean`:

```
theorem FCA_chap_B_4_1_4
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [CompleteSpace E]
    {Ω C : Set E} {f : E → ℝ}
    (hΩ : IsOpen Ω) (hf : DifferentiableOn ℝ f Ω)
    (hCΩ : C ⊆ Ω) (hC : Convex ℝ C) :
    (ConvexOn ℝ C f ↔
      ∀ x ∈ C, ∀ y ∈ C,
        0 ≤ ⟪gradient f x - gradient f y, x - y⟫_ℝ) ∧
    ...
```

Second stage (`BC_chap_20_25`).

`LeanWorkspace/Defs.lean`:

```
def MonotoneOperator {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℝ H]
    (A : H → Set H) : Prop :=
  ∀ x y : H, ∀ u ∈ A x, ∀ v ∈ A y, 0 ≤ ⟪x - y, u - v⟫_ℝ
```

GPT-6 Astra, Codex, attempt 2

First stage (`FCA_chap_B_4_1_4`).

`LeanWorkspace/Defs.lean` contains only `import Mathlib`.

`LeanWorkspace/FCA_chap_B_4_1_4.lean`:

```
theorem FCA_chap_B_4_1_4
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [CompleteSpace E]
    {f : E → ℝ} {Ω C : Set E}
    (hΩ : IsOpen Ω) (hC : Convex ℝ C) (hCΩ : C ⊆ Ω)
    (hf : DifferentiableOn ℝ f Ω) :
    (ConvexOn ℝ C f ↔
      ∀ x ∈ C, ∀ y ∈ C,
        0 ≤ ⟪gradient f x - gradient f y, x - y⟫_ℝ) ∧
    ...
```

Second stage (BC_chap_20_25).

LeanWorkspace/Defs.lean:

```
def IsMonotoneOperator (A : H → Set H) : Prop :=
  ∀ {|x y u v : H|}, u ∈ A x → v ∈ A y → 0 ≤ ⟪x - y, u - v⟫_ℝ
```

Claude Fable 5.1, Claude Code, attempt 1

First stage (FCA_chap_B_4_1_4).

LeanWorkspace/Defs.lean:

```
/-! ...
Here we add the monotonicity notions for (single-valued) mappings on a subset of a real
inner product space, following Hiriart-Urruty-Lemaréchal, *Fundamentals of Convex Analysis*,
Definition B.4.1.3 (compare Bauschke-Combettes, Definitions 20.1 and 22.1, and
Rockafellar-Wets, Definition 12.1, for the set-valued versions).
-/

def MonotoneMapOn (s : Set E) (F : E → E) : Prop :=
  ∀ {|x|}, x ∈ s → ∀ {|y|}, y ∈ s → 0 ≤ ⟪F x - F y, x - y⟫_ℝ
```

LeanWorkspace/FCA_chap_B_4_1_4.lean:

```
theorem FCA_chap_B_4_1_4 {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [CompleteSpace E] {Ω C : Set E} (hΩ : IsOpen Ω) (hC : Convex ℝ C) (hCΩ : C ⊆ Ω)
    {f : E → ℝ} (hf : DifferentiableOn ℝ f Ω) :
    (ConvexOn ℝ C f ↔ MonotoneMapOn C (∇ f)) ∧
    ...
```

Second stage (BC_chap_20_25).

LeanWorkspace/Defs.lean:

```
def MonotoneOperator (A : E → Set E) : Prop :=
  ∀ {|x u|}, u ∈ A x → ∀ {|y v|}, v ∈ A y → 0 ≤ ⟪x - y, u - v⟫_ℝ
```

Claude Fable 5.1, Claude Code, attempt 2

First stage (FCA_chap_B_4_1_4).

LeanWorkspace/Defs.lean:

```
def IsMonotoneOn (F : E → E) (C : Set E) : Prop :=
  ∀ x ∈ C, ∀ y ∈ C, 0 ≤ inner ℝ (F x - F y) (x - y)
```

LeanWorkspace/FCA_chap_B_4_1_4.lean:

```
theorem FCA_chap_B_4_1_4
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [CompleteSpace E]
    (f : E → ℝ) (Ω C : Set E)
    (hΩ : IsOpen Ω) (hf : DifferentiableOn ℝ f Ω) (hC : Convex ℝ C) (hCΩ : C ⊆ Ω) :
    (ConvexOn ℝ C f ↔ IsMonotoneOn (gradient f) C) ∧
    ...
```

Second stage (BC_chap_20_25).

LeanWorkspace/Defs.lean:

```
def IsMonotoneOperator (A : E → Set E) : Prop :=
  ∀ {|x u|}, u ∈ A x → ∀ {|y v|}, v ∈ A y → 0 ≤ inner ℝ (x - y) (u - v)
```

Claude Opus 5.5, Claude Code, attempt 1

First stage (FCA_chap_B_4_1_4).

LeanWorkspace/Defs.lean contains only `import Mathlib`.

LeanWorkspace/FCA_chap_B_4_1_4.lean:

```
theorem FCA_chap_B_4_1_4 {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [CompleteSpace E] {f : E → ℝ} {Ω C : Set E} (hΩ : IsOpen Ω)
    (hf : DifferentiableOn ℝ f Ω) (hC : Convex ℝ C) (hCΩ : C ⊆ Ω) :
    (ConvexOn ℝ C f ↔
      ∀ x ∈ C, ∀ x' ∈ C, 0 ≤ ⟪∇ f x - ∇ f x', x - x'⟫_ℝ) ∧
    ...
```

Second stage (BC_chap_20_25).

LeanWorkspace/Defs.lean:

```
def MonotoneOperator (A : H → Set H) : Prop :=
  ∀ {|x u|}, u ∈ A x → ∀ {|y v|}, v ∈ A y → 0 ≤ ⟪x - y, u - v⟫_ℝ
```

Claude Opus 5.5, Claude Code, attempt 2

First stage (FCA_chap_B_4_1_4).

LeanWorkspace/Defs.lean contains only `import Mathlib`.

LeanWorkspace/FCA_chap_B_4_1_4.lean:

```
theorem FCA_chap_B_4_1_4
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [CompleteSpace E]
    {f : E → ℝ} {Ω C : Set E} (hΩ : IsOpen Ω) (hf : DifferentiableOn ℝ f Ω)
    (hC : Convex ℝ C) (hCΩ : C ⊆ Ω) :
    (ConvexOn ℝ C f ↔
      ∀ x ∈ C, ∀ x' ∈ C, 0 ≤ ⟪∇ f x - ∇ f x', x - x'⟫_ℝ) ∧
    ...
```

Second stage (BC_chap_20_25).

LeanWorkspace/Defs.lean:

```
def IsMonotoneOperator (A : H → Set H) : Prop :=
  ∀ {|x u y v : H|}, u ∈ A x → v ∈ A y → 0 ≤ ⟪x - y, u - v⟫_ℝ
```

G.1.2 Detailed prompt

GPT-6 Astra, Codex, attempt 1

First stage (FCA_chap_B_4_1_4).

LeanWorkspace/Defs.lean:

```
def IsMonotoneOperator {E : Type*} [NormedAddCommGroup E]
    [InnerProductSpace ℝ E] [CompleteSpace E] (T : E → Set E) : Prop :=
  ∀ {|x₀ x₁ : E|}, ∀ v₀ ∈ T x₀, ∀ v₁ ∈ T x₁,
    0 ≤ inner ℝ (v₁ - v₀) (x₁ - x₀)
```

LeanWorkspace/FCA_chap_B_4_1_4.lean:

```
theorem FCA_chap_B_4_1_4
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [FiniteDimensional ℝ E]
    (f : E → ℝ) (Ω C : Set E) (c : ℝ)
    -- Constraint set and open neighborhood.
    (hC_nonempty : C.Nonempty) (hC_convex : Convex ℝ C)
    (hΩ_open : IsOpen Ω) (hC_subset : C ⊆ Ω)
    -- Function.
    (hf_differentiable : DifferentiableOn ℝ f Ω)
    -- Modulus.
    (hc_pos : 0 < c) :
    let gradientOnC : E → Set E := fun x => {v | x ∈ C ∧ v = gradient f x}
    (ConvexOn ℝ C f ↔ IsMonotoneOperator gradientOnC) ∧
    ...
```

Second stage (BC_chap_20_25).

LeanWorkspace/Defs.lean:

```
def IsMaximallyMonotoneOperator {E : Type*} [NormedAddCommGroup E]
    [InnerProductSpace ℝ E] [CompleteSpace E] (T : E → Set E) : Prop :=
  Maximal IsMonotoneOperator T
```

GPT-6 Astra, Codex, attempt 2

First stage (FCA_chap_B_4_1_4).

LeanWorkspace/Defs.lean:

```
def IsMonotoneOperator (T : E → Set E) : Prop :=
  ∀ {|x₀ x₁ v₀ v₁ : E|}, v₀ ∈ T x₀ → v₁ ∈ T x₁ →
    0 ≤ inner ℝ (v₁ - v₀) (x₁ - x₀)
```

LeanWorkspace/FCA_chap_B_4_1_4.lean:

```
theorem FCA_chap_B_4_1_4
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [FiniteDimensional ℝ E]
    (Ω C : Set E) (f : E → ℝ) (c : ℝ)
    -- Domain and constraint set.
    (hΩ_open : IsOpen Ω)
    (hC_nonempty : C.Nonempty)
    (hC_convex : Convex ℝ C)
    (hC_subset : C ⊆ Ω)
    -- Differentiability on the open neighborhood of C.
    (hf_differentiable : DifferentiableOn ℝ f Ω)
    -- Modulus for the strong equivalence.
    (hc_pos : 0 < c) :
    let gradOnC : E → Set E := fun x => {v | x ∈ C ∧ v = gradient f x}
    (ConvexOn ℝ C f ↔ IsMonotoneOperator gradOnC) ∧
    ...
```

Second stage (BC_chap_20_25).

LeanWorkspace/Defs.lean:

```
def IsMaximallyMonotoneOperator (T : E → Set E) : Prop :=
  Maximal IsMonotoneOperator T
```

Claude Fable 5.1, Claude Code, attempt 1

First stage (FCA_chap_B_4_1_4).

LeanWorkspace/Defs.lean:

```
def toMultifunOn (F : E → E) (C : Set E) : E → Set E :=
  fun x => {v | x ∈ C ∧ v = F x}

def IsMonotoneOperator (T : E → Set E) : Prop :=
  ∀ {|x₀ x₁ : E|}, ∀ v₀ ∈ T x₀, ∀ v₁ ∈ T x₁, 0 ≤ ⟪v₁ - v₀, x₁ - x₀⟫_ℝ
```

LeanWorkspace/FCA_chap_B_4_1_4.lean:

```
theorem FCA_chap_B_4_1_4
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → ℝ) (Ω C : Set E) (c : ℝ)
    -- the function: differentiable on the open set Ω
    (hΩ_open : IsOpen Ω) (hf_diff : DifferentiableOn ℝ f Ω)
    -- the set: C is a nonempty convex subset of Ω
    (hC_nonempty : C.Nonempty) (hC_convex : Convex ℝ C) (hC_sub : C ⊆ Ω)
    -- the modulus of strong convexity / strong monotonicity
    (hc_pos : 0 < c) :
    (ConvexOn ℝ C f ↔ IsMonotoneOperator (toMultifunOn (gradient f) C)) ∧
    ...
```

Second stage (BC_chap_20_25).

LeanWorkspace/Defs.lean:

```
def IsMaximallyMonotoneOperator (T : E → Set E) : Prop :=
  Maximal IsMonotoneOperator T
```

Claude Fable 5.1, Claude Code, attempt 2

First stage (FCA_chap_B_4_1_4).

LeanWorkspace/Defs.lean:

```
def Function.toMultifunOn (g : E → F) (C : Set E) : E → Set F :=
  fun x => {v | x ∈ C ∧ v = g x}

def IsMonotoneMultifun (T : E → Set E) : Prop :=
  ∀ x₀ x₁ : E, ∀ v₀ ∈ T x₀, ∀ v₁ ∈ T x₁, 0 ≤ inner ℝ (v₁ - v₀) (x₁ - x₀)
```

LeanWorkspace/FCA_chap_B_4_1_4.lean:

```
theorem FCA_chap_B_4_1_4
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    -- data: the function, the open set and the convex set
    (f : E → ℝ) (Ω C : Set E)
    -- the modulus of strong convexity / strong monotonicity
    (c : ℝ)
    -- the set `C`: nonempty, convex, contained in `Ω`
    (hC_nonempty : C.Nonempty) (hC_convex : Convex ℝ C) (hC_sub : C ⊆ Ω)
    -- the function: differentiable on the open set `Ω`
    (hΩ_open : IsOpen Ω) (hf_diff : DifferentiableOn ℝ f Ω)
    -- the modulus is positive
    (hc_pos : 0 < c) :
    (ConvexOn ℝ C f ↔ IsMonotoneMultifun ((gradient f).toMultifunOn C)) ∧
    ...
```

Second stage (BC_chap_20_25).

LeanWorkspace/Defs.lean:

```
def IsMaximallyMonotoneMultifun (T : E → Set E) : Prop :=
  IsMonotoneMultifun T ∧
    ∀ S : E → Set E, IsMonotoneMultifun S → (∀ x, T x ⊆ S x) → ∀ x, S x ⊆ T x
```

Claude Opus 5.5, Claude Code, attempt 1

First stage (FCA_chap_B_4_1_4).

LeanWorkspace/Defs.lean:

```
def IsMonotoneOperator (T : E → Set E) : Prop :=
  ∀ {|x₀ x₁ v₀ v₁ : E|}, v₀ ∈ T x₀ → v₁ ∈ T x₁ → 0 ≤ ⟪v₁ - v₀, x₁ - x₀⟫_ℝ
```

LeanWorkspace/FCA_chap_B_4_1_4.lean:

```
theorem FCA_chap_B_4_1_4
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → ℝ) (Ω C : Set E) (c : ℝ)
    -- the open set `Ω` on which `f` is differentiable
    (hΩ_open : IsOpen Ω) (hf_diff : DifferentiableOn ℝ f Ω)
    -- the set `C`: nonempty, convex, contained in `Ω`
    (hC_nonempty : C.Nonempty) (hC_convex : Convex ℝ C) (hC_sub : C ⊆ Ω)
    -- the modulus of strong convexity / strong monotonicity
    (hc : 0 < c) :
    (ConvexOn ℝ C f ↔
      IsMonotoneOperator (fun x => {v | x ∈ C ∧ v = gradient f x})) ∧
    ...
```

Second stage (BC_chap_20_25).

LeanWorkspace/Defs.lean:

```
def IsMaximallyMonotoneOperator (T : E → Set E) : Prop :=
  IsMonotoneOperator T ∧
    ∀ S : E → Set E, IsMonotoneOperator S → (∀ x, T x ⊆ S x) → ∀ x, S x = T x
```

Claude Opus 5.5, Claude Code, attempt 2

First stage (FCA_chap_B_4_1_4).

LeanWorkspace/Defs.lean:

```
def IsMonotoneOperator (T : E → Set E) : Prop :=
  ∀ {|x₀ x₁ v₀ v₁ : E|}, v₀ ∈ T x₀ → v₁ ∈ T x₁ → 0 ≤ inner ℝ (v₁ - v₀) (x₁ - x₀)
```

LeanWorkspace/FCA_chap_B_4_1_4.lean:

```
theorem FCA_chap_B_4_1_4
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → ℝ) (Ω C : Set E)
    -- `f` is differentiable on the open set `Ω`
    (hΩ_open : IsOpen Ω) (hf_diff : DifferentiableOn ℝ f Ω)
    -- `C` is a nonempty convex subset of `Ω`
    (hC_nonempty : C.Nonempty) (hC_convex : Convex ℝ C) (hC_sub : C ⊆ Ω) :
    (ConvexOn ℝ C f ↔
      IsMonotoneOperator (fun x => {v | x ∈ C ∧ v = gradient f x})) ∧
    ...
```

Second stage (BC_chap_20_25).

LeanWorkspace/Defs.lean:

```
def IsMaximalMonotoneOperator (T : E → Set E) : Prop :=
  Maximal IsMonotoneOperator T
```

G.2 *Fundamentals of Convex Analysis*, Chapter B, Proposition 1.2.1 → *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*, Theorem 9.20 (FCA_chap_B_1_2_1 → BC_chap_9_20)

Under the basic prompt, GPT-6 Astra reused the epigraph definition of convexity for both Proposition 1.2.1 and Theorem 9.20. In one attempt, Fable 5.1 formalized Proposition 1.2.1 using ConvexOn

for real-valued functions with a domain. It separately defined convexity for extended real-valued functions to formalize Theorem 9.20, but did not update the earlier formalization. In another attempt, Fable 5.1 defined and used the inequality characterization of convexity for both textbook statements. Opus 5.5 inlined the inequality definition of convexity for Proposition 1.2.1, then separately inlined the epigraph definition for Theorem 9.20.

Under the detailed prompt, all three agents used the epigraph definition of convexity for extended real-valued functions for both textbook results.

G.2.1 Basic prompt

GPT-6 Astra, Codex, attempt 1

First stage (`FCA_chap_B_1_2_1`).

`LeanWorkspace/Defs.lean`:

```
def IsProper {E : Type*} (f : E → EReal) : Prop :=
  (∀ x, f x ≠ ⊥) ∧ (effectiveDomain f).Nonempty

def epigraph {E : Type*} (f : E → EReal) : Set (E × ℝ) :=
  {p | f p.1 ≤ (p.2 : EReal)}
```

`LeanWorkspace/FCA_chap_B_1_2_1.lean`:

```
theorem FCA_chap_B_1_2_1
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [FiniteDimensional ℝ E] (f : E → EReal)
    (hf_proper : ConvexAnalysis.IsProper f)
    (hf_convex : Convex ℝ (ConvexAnalysis.epigraph f)) :
    ...
```

Second stage (`BC_chap_9_20`).

`LeanWorkspace/Defs.lean` is unchanged from the first stage.

`LeanWorkspace/BC_chap_9_20.lean`:

```
theorem BC_chap_9_20
    {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℝ H]
    [CompleteSpace H] (f : H → EReal)
    (hf_proper : ConvexAnalysis.IsProper f)
    (hf_lsc : LowerSemicontinuous f)
    (hf_convex : Convex ℝ (ConvexAnalysis.epigraph f)) :
    ...
```

GPT-6 Astra, Codex, attempt 2

First stage (`FCA_chap_B_1_2_1`).

`LeanWorkspace/Defs.lean`:

```
def IsProperConvex {E : Type*} [AddCommGroup E] [Module ℝ E]
    (f : E → EReal) : Prop :=
  (∀ x, f x ≠ ⊥) ∧ (effectiveDomain f).Nonempty ∧ Convex ℝ (epigraph f)
```

`LeanWorkspace/FCA_chap_B_1_2_1.lean`:

```
theorem FCA_chap_B_1_2_1
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [FiniteDimensional ℝ E] (f : E → EReal) (hf : IsProperConvex f) :
    ...
```

Second stage (`BC_chap_9_20`).

`LeanWorkspace/Defs.lean` is unchanged from the first stage.

`LeanWorkspace/BC_chap_9_20.lean`:

```
theorem BC_chap_9_20
    {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℝ H] [CompleteSpace H]
    (f : H → EReal) (hf : IsProperConvex f) (hlsc : LowerSemicontinuous f) :
    ...
```

Claude Fable 5.1, Claude Code, attempt 1

First stage (`FCA_chap_B_1_2_1`).

`LeanWorkspace/Defs.lean` contains only `import Mathlib`.

`LeanWorkspace/FCA_chap_B_1_2_1.lean`:

```
theorem FCA_chap_B_1_2_1
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    {D : Set E} {f : E → ℝ} (hD : D.Nonempty) (hf : ConvexOn ℝ D f) :
    ...
```

Second stage (`BC_chap_9_20`).

`LeanWorkspace/Defs.lean` contains only `import Mathlib`.

`LeanWorkspace/BC_chap_9_20.lean`:

```
theorem BC_chap_9_20
    {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℝ H] [CompleteSpace H]
    {f : H → EReal} (hbot : ∀ x, f x ≠ ⊥) (hdom : ∃ x, f x ≠ ⊤)
    (hlsc : LowerSemicontinuous f) (hconv : Convex ℝ {q : H × ℝ | f q.1 ≤ (q.2 : EReal)}) :
    ...
```

Claude Fable 5.1, Claude Code, attempt 2

First stage (`FCA_chap_B_1_2_1`).

`LeanWorkspace/Defs.lean`:

```
def IsConvexFunction (f : E → EReal) : Prop :=
  ∀ x x' : E, ∀ a : ℝ, 0 < a → a < 1 →
    f (a • x + (1 - a) • x') ≤ (a : EReal) * f x + ((1 - a : ℝ) : EReal) * f x'

structure IsProperConvexFunction (f : E → EReal) : Prop where
  /-- `f` never takes the value `-∞`. -/
  ne_bot : ∀ x, f x ≠ ⊥
  /-- `f` is not identically `+∞`, i.e. `dom f ≠ ∅`. -/
  exists_ne_top : ∃ x, f x ≠ ⊤
  /-- `f` satisfies Jensen's inequality. -/
  convex : IsConvexFunction f
```

`LeanWorkspace/FCA_chap_B_1_2_1.lean`:

```
theorem FCA_chap_B_1_2_1 {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [FiniteDimensional ℝ E] {f : E → EReal} (hf : IsProperConvexFunction f) :
    ...
```

Second stage (`BC_chap_9_20`).

`LeanWorkspace/Defs.lean`:

```
structure IsProperLscConvexFunction (f : E → EReal) : Prop extends IsProperConvexFunction f where
  /-- `f` is lower semicontinuous. -/
  lowerSemicontinuous : LowerSemicontinuous f
```

`LeanWorkspace/BC_chap_9_20.lean`:

```
theorem BC_chap_9_20 {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℝ H]
    [CompleteSpace H] {f : H → EReal} (hf : IsProperLscConvexFunction f) :
    ...
```

Claude Opus 5.5, Claude Code, attempt 1

First stage (FCA_chap_B_1_2_1).

LeanWorkspace/Defs.lean contains only import Mathlib.

LeanWorkspace/FCA_chap_B_1_2_1.lean:

```
theorem FCA_chap_B_1_2_1 {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [FiniteDimensional ℝ E] (f : E → EReal)
    (hf_ne_bot : ∀ x, f x ≠ ⊥)
    (hf_ne_top : ∃ x, f x ≠ ⊤)
    (hf_conv : ∀ x x' : E, ∀ α : ℝ, 0 < α → α < 1 →
      f (α • x + (1 - α) • x') ≤ (α : EReal) * f x + ((1 - α : ℝ) : EReal) * f x') :
    ...
```

Second stage (BC_chap_9_20).

LeanWorkspace/Defs.lean contains only import Mathlib.

LeanWorkspace/BC_chap_9_20.lean:

```
theorem BC_chap_9_20 {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℝ H]
    [CompleteSpace H] (f : H → EReal)
    (hf_ne_bot : ∀ x, f x ≠ ⊥)
    (hf_dom : ∃ x, f x < ⊤)
    (hf_conv : Convex ℝ {q : H × ℝ | f q.1 ≤ q.2})
    (hf_lsc : LowerSemicontinuous f) :
    ...
```

Claude Opus 5.5, Claude Code, attempt 2

First stage (FCA_chap_B_1_2_1).

LeanWorkspace/Defs.lean contains only import Mathlib.

LeanWorkspace/FCA_chap_B_1_2_1.lean:

```
theorem FCA_chap_B_1_2_1
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → EReal)
    (hf_ne_bot : ∀ x, f x ≠ ⊥)
    (hf_proper : ∃ x, f x ≠ ⊤)
    (hf_conv : ∀ x x' : E, ∀ α : ℝ, 0 < α → α < 1 →
      f (α • x + (1 - α) • x') ≤ (α : EReal) * f x + ((1 - α : ℝ) : EReal) * f x') :
    ...
```

Second stage (BC_chap_9_20).

LeanWorkspace/Defs.lean contains only import Mathlib.

LeanWorkspace/BC_chap_9_20.lean:

```
theorem BC_chap_9_20
    {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℝ H] [CompleteSpace H]
    (f : H → EReal)
    (hf_ne_bot : ∀ x, f x ≠ ⊥)
    (hf_proper : ∃ x, f x < ⊤)
    (hf_lsc : LowerSemicontinuous f)
    (hf_conv : Convex ℝ {q : H × ℝ | f q.1 ≤ (q.2 : EReal)}) :
    ...
```

G.2.2 Detailed prompt

GPT-6 Astra, Codex, attempt 1

First stage (FCA_chap_B_1_2_1).

LeanWorkspace/Defs.lean:

```
def IsProperEReal {E : Type*} (f : E → EReal) : Prop :=
  (∀ x, ⊥ < f x) ∧ (effDom f).Nonempty

def ConvexEReal {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    (f : E → EReal) : Prop :=
  Convex ℝ (epigraph f)
```

LeanWorkspace/FCA_chap_B_1_2_1.lean:

```
theorem FCA_chap_B_1_2_1
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- Function hypotheses: f belongs to the source's class Conv ℝ^n.
    (hf_proper : IsProperEReal f) (hf_convex : ConvexEReal f) :
    (∃ a : E →ᵃ[ℝ] ℝ, ∀ x : E, (a x : EReal) ≤ f x) ∧
    ...
```

Second stage (BC_chap_9_20).

LeanWorkspace/Defs.lean:

```
def IsAffineMinorant {E : Type*} [AddCommGroup E] [Module ℝ E]
    (f : E → EReal) (a : E →ᵃ[ℝ] ℝ) : Prop :=
  ∀ x : E, (a x : EReal) ≤ f x
```

LeanWorkspace/FCA_chap_B_1_2_1.lean (edited):

```
theorem FCA_chap_B_1_2_1
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- Function hypotheses: f belongs to the source's class Conv ℝ^n.
    (hf_proper : IsProperEReal f) (hf_convex : ConvexEReal f) :
    (∃ a : E →ᵃ[ℝ] ℝ, IsAffineMinorant f a) ∧
    ...
```

LeanWorkspace/BC_chap_9_20.lean:

```
theorem BC_chap_9_20
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [CompleteSpace E]
    (f : E → EReal)
    -- Function hypotheses: f ∈ Γ₀(H), as in Definition 9.12.
    (hf_proper : IsProperEReal f)
    (hf_lowerSemicontinuous : LowerSemicontinuous f)
    (hf_convex : ConvexEReal f) :
    (∃ a : E →ᴬ[ℝ] ℝ, IsAffineMinorant f a.toAffineMap) ∧
    ...
```

GPT-6 Astra, Codex, attempt 2

First stage (FCA_chap_B_1_2_1).

LeanWorkspace/Defs.lean:

```
def IsProper {X : Type*} (f : X → EReal) : Prop :=
  (∀ x, ⊥ < f x) ∧ (effDom f).Nonempty

def ConvexEReal {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    (f : E → EReal) : Prop :=
  Convex ℝ (epigraph f)
```

`LeanWorkspace/FCA_chap_B_1_2_1.lean`:

```
theorem FCA_chap_B_1_2_1
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- Function: membership in the source's class `Conv ℝ^n`.
    (hf_proper : IsProper f)
    (hf_convex : ConvexEReal f) :
    (∃ a : E →ᵃ[ℝ] ℝ, ∀ x : E, (a x : EReal) ≤ f x) ∧
      (∀ x₀ ∈ intrinsicInterior ℝ (effDom f),
        ∃ s ∈ (affineSpan ℝ (effDom f)).direction,
          ∀ x : E, f x₀ + (inner ℝ s (x - x₀) : EReal) ≤ f x) := by
  sorry
```

Second stage (`BC_chap_9_20`).

`LeanWorkspace/Defs.lean`:

```
def liftRealtoEReal {X : Type*} (f : X → ℝ) : X → EReal :=
  fun x => (f x : EReal)

def convexSubdifferential {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    (f : E → EReal) (x : E) : Set E :=
  {u | ⊥ < f x ∧ f x < ⊤ ∧
    ∀ y : E, (inner ℝ (y - x) u : EReal) + f x ≤ f y}
```

`LeanWorkspace/FCA_chap_B_1_2_1.lean` (edited):

```
theorem FCA_chap_B_1_2_1
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- Function: membership in the source's class `Conv ℝ^n`.
    (hf_proper : IsProper f)
    (hf_convex : ConvexEReal f) :
    (∃ a : E →ᵃ[ℝ] ℝ, liftRealtoEReal a ≤ f) ∧
      (∀ x₀ ∈ intrinsicInterior ℝ (effDom f),
        ∃ s ∈ (affineSpan ℝ (effDom f)).direction,
          s ∈ convexSubdifferential f x₀) := by
  sorry
```

`LeanWorkspace/BC_chap_9_20.lean`:

```
theorem BC_chap_9_20
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [CompleteSpace E]
    (f : E → EReal)
    -- Function: membership in the source's class Γ₀(H).
    (hf_proper : IsProper f)
    (hf_lowerSemicontinuous : LowerSemicontinuous f)
    (hf_convex : ConvexEReal f) :
    (∃ a : E →ᴬ[ℝ] ℝ, liftRealtoEReal a ≤ f) ∧
      (∃ p ∈ effDom f, ∃ u : E, u ∈ convexSubdifferential f p) := by
  sorry
```

Claude Fable 5.1, Claude Code, attempt 1

First stage (`FCA_chap_B_1_2_1`).

`LeanWorkspace/Defs.lean`:

```
def ProperEReal (f : E → EReal) : Prop := (∀ x, ⊥ < f x) ∧ (effDom f).Nonempty

def ConvexEReal (f : E → EReal) : Prop := Convex ℝ (epigraph f)
```

`LeanWorkspace/FCA_chap_B_1_2_1.lean`:

```
theorem FCA_chap_B_1_2_1
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- function: `f ∈ Conv ℝ^n`, i.e. `f : ℝ^n → ℝ ∪ {+∞}` not identically `+∞` (proper)
    -- with convex epigraph
    (hf_proper : ProperEReal f)
    (hf_convex : ConvexEReal f) :
    -- (1) `f` is minorized by some affine function `x ↦ ⟨s, x⟩ − b`
    (∃ (s : E) (b : ℝ), ∀ x, ((inner ℝ s x - b : ℝ) : EReal) ≤ f x) ∧
    ...
```

Second stage (`BC_chap_9_20`).

`LeanWorkspace/Defs.lean`:

```
def HasContinuousAffineMinorant (f : E → EReal) : Prop :=
  ∃ (u : E) (η : ℝ), ∀ x, ((inner ℝ u x + η : ℝ) : EReal) ≤ f x
```

`LeanWorkspace/FCA_chap_B_1_2_1.lean` (edited):

```
theorem FCA_chap_B_1_2_1
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- function: `f ∈ Conv ℝ^n`, i.e. `f : ℝ^n → ℝ ∪ {+∞}` not identically `+∞` (proper)
    -- with convex epigraph
    (hf_proper : ProperEReal f)
    (hf_convex : ConvexEReal f) :
    -- (1) `f` is minorized by some affine function `x ↦ ⟨s, x⟩ − b`
    HasContinuousAffineMinorant f ∧
    ...
```

`LeanWorkspace/BC_chap_9_20.lean`:

```
theorem BC_chap_9_20
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [CompleteSpace E]
    (f : E → EReal)
    -- function: `f ∈ Γ_0(H)`, i.e. `f : H → ]−∞, +∞]` is proper, lower semicontinuous
    -- and convex (Definition 9.12)
    (hf_proper : ProperEReal f)
    (hf_lsc : LowerSemicontinuous f)
    (hf_convex : ConvexEReal f) :
    -- `f` possesses a continuous affine minorant `⟨· | u⟩ + η`
    HasContinuousAffineMinorant f ∧
    ...
```

Claude Fable 5.1, Claude Code, attempt 2

First stage (`FCA_chap_B_1_2_1`).

`LeanWorkspace/Defs.lean`:

```
def IsProperEReal {E : Type*} (f : E → EReal) : Prop :=
  (∀ x, ⊥ < f x) ∧ (effDom f).Nonempty

def ConvexEReal {E : Type*} [AddCommGroup E] [Module ℝ E] (f : E → EReal) : Prop :=
  Convex ℝ (epigraph f)
```

`LeanWorkspace/FCA_chap_B_1_2_1.lean`:

```
theorem FCA_chap_B_1_2_1
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- function: `f ∈ Conv ℝ^n`, i.e. proper (values in `ℝ ∪ {+∞}`, not identically `+∞`)
    -- and convex
    (hf_proper : IsProperEReal f)
    (hf_convex : ConvexEReal f) :
    -- (1) `f` is minorized by an affine function
    (∃ a : E →ᵃ[ℝ] ℝ, ∀ x, (a x : EReal) ≤ f x) ∧
    ...
```

Second stage (BC_chap_9_20).

LeanWorkspace/Defs.lean:

```
def HasContinuousAffineMinorant {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    (f : E → EReal) : Prop :=
  ∃ a : E →ᴬ[ℝ] ℝ, ∀ x, (a x : EReal) ≤ f x
```

LeanWorkspace/FCA_chap_B_1_2_1.lean (edited):

```
theorem FCA_chap_B_1_2_1
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- function: `f ∈ Conv ℝ^n`, i.e. proper (values in `ℝ ∪ {+∞}`, not identically `+∞`)
    -- and convex
    (hf_proper : IsProperEReal f)
    (hf_convex : ConvexEReal f) :
    -- (1) `f` is minorized by an affine function `ℝ^n → ℝ` (every affine map on a
    -- finite-dimensional space is continuous, so this is `HasContinuousAffineMinorant f`)
    ...
```

LeanWorkspace/BC_chap_9_20.lean:

```
theorem BC_chap_9_20
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [CompleteSpace E]
    (f : E → EReal)
    -- function: `f ∈ Γ₀(H)`, i.e. proper, lower semicontinuous and convex
    (hf_proper : IsProperEReal f)
    (hf_lsc : LowerSemicontinuous f)
    (hf_convex : ConvexEReal f) :
    -- `f` possesses a continuous affine minorant
    HasContinuousAffineMinorant f ∧
    ...
```

Claude Opus 5.5, Claude Code, attempt 1

First stage (FCA_chap_B_1_2_1).

LeanWorkspace/Defs.lean:

```
def ProperEReal (f : E → EReal) : Prop :=
  (∀ x, ⊥ < f x) ∧ (effDom f).Nonempty

def ConvexEReal [AddCommMonoid E] [Module ℝ E] (f : E → EReal) : Prop :=
  Convex ℝ (epigraph f)
```

LeanWorkspace/FCA_chap_B_1_2_1.lean:

```
theorem FCA_chap_B_1_2_1
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- the function: `f ∈ Conv ℝ^n` (values in `ℝ ∪ {+∞}`, not identically `+∞`, convex)
    (hf_proper : ProperEReal f) (hf_convex : ConvexEReal f) :
    ...
```

Second stage (BC_chap_9_20).

LeanWorkspace/Defs.lean: no new definitions for the shared notion.

LeanWorkspace/BC_chap_9_20.lean:

```
theorem BC_chap_9_20
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [CompleteSpace E]
    (f : E → EReal)
    -- the function: `f ∈ Γ₀(H)` (proper, lower semicontinuous, convex)
    (hf_proper : ProperEReal f) (hf_lsc : LowerSemicontinuous f) (hf_convex : ConvexEReal f) :
    ...
```

Claude Opus 5.5, Claude Code, attempt 2

First stage (FCA_chap_B_1_2_1).

LeanWorkspace/Defs.lean:

```
def ProperEReal {E : Type*} (f : E → EReal) : Prop :=
  (∀ x, ⊥ < f x) ∧ (effDom f).Nonempty

def ConvexEReal {E : Type*} [AddCommMonoid E] [Module ℝ E] (f : E → EReal) : Prop :=
  Convex ℝ (epigraph f)
```

LeanWorkspace/FCA_chap_B_1_2_1.lean:

```
theorem FCA_chap_B_1_2_1 {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- the function: `f ∈ Conv ℝ^n`, i.e. proper (values in `ℝ ∪ {+∞}`, not identically `+∞`)
    -- and convex
    (hf_proper : ProperEReal f) (hf_convex : ConvexEReal f) :
    -- `f` is minorized by some affine function
    (∃ a : E →ᵃ[ℝ] ℝ, ∀ x, (a x : EReal) ≤ f x) ∧
    ...
```

Second stage (BC_chap_9_20).

LeanWorkspace/Defs.lean:

```
def HasContinuousAffineMinorant {E : Type*} [AddCommGroup E] [Module ℝ E] [TopologicalSpace E]
    (f : E → EReal) : Prop :=
  ∃ a : E →ᴬ[ℝ] ℝ, ∀ x, ((a x : ℝ) : EReal) ≤ f x
```

LeanWorkspace/FCA_chap_B_1_2_1.lean (edited):

```
theorem FCA_chap_B_1_2_1 {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- the function: `f ∈ Conv ℝ^n`, i.e. proper (values in `ℝ ∪ {+∞}`, not identically `+∞`)
    -- and convex
    (hf_proper : ProperEReal f) (hf_convex : ConvexEReal f) :
    -- `f` is minorized by some affine function (affine functions on `E` are continuous)
    HasContinuousAffineMinorant f ∧
    ...
```

LeanWorkspace/BC_chap_9_20.lean:

```
theorem BC_chap_9_20 {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [CompleteSpace E]
    (f : E → EReal)
    -- the function: `f ∈ Γ₀(H)`, i.e. proper, lower semicontinuous and convex
    (hf_proper : ProperEReal f) (hf_lsc : LowerSemicontinuous f) (hf_convex : ConvexEReal f) :
    -- `f` possesses a continuous affine minorant
    HasContinuousAffineMinorant f ∧
    ...
```

G.3 *Fundamentals of Convex Analysis*, Chapter B, Proposition 3.2.4, second part → *Variational Analysis*, Theorem 3.26 (`FCA_chap_B_3_2_4_ii` → `RW_chap_3_26`)

Under basic prompt, GPT-6 Astra first defined the asymptotic function using the quotient formula, then separately defined the horizon function without updating the earlier definition. In one attempt, Fable 5.1 used the epigraph definition of the asymptotic function, which coincides with the horizon function in Rockafellar and Wets' *Variational Analysis* definition, thus unifying both textbook formalizations. In another attempt, Fable 5.1 introduced separate definitions. Opus 5.5 likewise defined the quotient version of the asymptotic function and the horizon function separately.

Under the detailed prompt, all three agents used the epigraph definition, which unifies both textbook definitions.

G.3.1 Basic prompt

GPT-6 Astra, Codex, attempt 1

First stage (`FCA_chap_B_3_2_4_ii`).

`LeanWorkspace/Defs.lean`:

```
noncomputable def asymptoticFunction [AddCommGroup E] [Module ℝ E]
    (f : E → EReal) (d : E) : EReal :=
  ⨆ (x : E) (_ : f x ≠ ⊥) (_ : f x ≠ ⊤) (t : ℝ) (_ : 0 < t),
    (f (x + t • d) - f x) / (t : EReal)
```

Second stage (`RW_chap_3_26`).

`LeanWorkspace/Defs.lean`:

```
noncomputable def horizonFunction [NormedAddCommGroup E] [NormedSpace ℝ E]
    (f : E → EReal) (d : E) : EReal := by
  classical
  exact if ∀ x, f x = ⊤ then
    if d = 0 then 0 else ⊤
  else
    Filter.liminf (fun p : ℝ × E => (p.1 : EReal) * f (p.1⁻¹ • p.2))
      (nhdsWithin (0 : ℝ) (Set.Ioi 0) ×ˢ nhds d)
```

`LeanWorkspace/RW_chap_3_26.lean`:

```
theorem RW_chap_3_26
    {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    [FiniteDimensional ℝ E] (f : E → EReal)
    (hproper : IsProper f) (hlsc : LowerSemicontinuous f) :
    (lowerLinearGrowthRate f =
      ⊓ (d : E) (_ : ‖d‖ = 1), horizonFunction f d) ∧
    ...
```

GPT-6 Astra, Codex, attempt 2

First stage (`FCA_chap_B_3_2_4_ii`).

`LeanWorkspace/Defs.lean`:

```
noncomputable def asymptoticFunction {E : Type*} [AddCommGroup E] [Module ℝ E]
    (f : E → EReal) (d : E) : EReal :=
  ⨆ (x : E) (_ : x ∈ effectiveDomain f) (t : ℝ) (_ : 0 < t),
    (f (x + t • d) - f x) / (t : EReal)
```

Second stage (`RW_chap_3_26`).

`LeanWorkspace/Defs.lean`:

```
/-- The horizon function, using the joint lower limit in Rockafellar--Wets,
formula 3(3): `liminf_{λ ↓ 0, x → d} λ f(λ⁻¹ • x)`.
For the identically `+∞` function, Definition 3.17 prescribes the indicator
of `{0}`. Unlike `asymptoticFunction`, this definition also applies to
nonconvex functions. -/
noncomputable def horizonFunction {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    (f : E → EReal) (d : E) : EReal := by
  classical
  exact if ∀ x, f x = ⊤ then
    if d = 0 then 0 else ⊤
  else
    Filter.liminf (fun p : ℝ × E => (p.1 : EReal) * f (p.1⁻¹ • p.2))
      (nhdsWithin (0 : ℝ) (Set.Ioi 0) ×ˢ nhds d)
```

LeanWorkspace/RW_chap_3_26.lean:

```
theorem RW_chap_3_26
    {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → EReal)
    (hproper : IsProper f)
    (hlsc : LowerSemicontinuous f) :
    (liminfNormRatio f = ⊓ (d : E) (_ : ‖d‖ = 1), horizonFunction f d) ∧
    ...
```

Claude Fable 5.1, Claude Code, attempt 1

First stage (FCA_chap_B_3_2_4_ii).

LeanWorkspace/Defs.lean:

```
noncomputable def asymptoticFun (f : E → EReal) (d : E) : EReal :=
  ⊓ (p : ℝ) (_ : (d, p) ∈ asymptoticCone ℝ (epigraph f)), (p : EReal)
```

Second stage (RW_chap_3_26).

LeanWorkspace/Defs.lean:

```
/-- ...

This is also the *horizon function* `f^∞` of Rockafellar-Wets, *Variational Analysis*, Def. 3.17
(`epi f^∞ = (epi f)^∞`), for every `f ≢ +∞`: Mathlib's `asymptoticCone` is the filter formulation
of the horizon cone `C^∞ = {x | ∃ x^ν ∈ C, λ^ν ↘ 0, λ^ν x^ν → x}` of R&W Def. 3.3, and the two agree
for nonempty `C`.  (For `f ≡ +∞`, R&W set `f^∞ = δ_{0}` by convention, whereas this definition
gives `+∞` everywhere; the two differ only at `0`.) -/
noncomputable def asymptoticFun (f : E → EReal) (d : E) : EReal :=
  ⊓ (p : ℝ) (_ : (d, p) ∈ asymptoticCone ℝ (epigraph f)), (p : EReal)
```

LeanWorkspace/RW_chap_3_26.lean:

```
theorem RW_chap_3_26
    {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → EReal) (hlsc : LowerSemicontinuous f)
    (hbot : ∀ x, f x ≠ ⊥) (hdom : ∃ x, f x ≠ ⊤) :
    -- formula 3(7)
    (liminfDivNorm f = ⊓ (x : E) (_ : ‖x‖ = 1), asymptoticFun f x) ∧
    ...
```

Claude Fable 5.1, Claude Code, attempt 2

First stage (FCA_chap_B_3_2_4_ii).

LeanWorkspace/Defs.lean:

```
noncomputable def asymptoticFunction (f : E → EReal) (d : E) : EReal :=
  ⨆ (x : E) (_ : x ∈ effectiveDomain f) (t : ℝ) (_ : 0 < t), (f (x + t • d) - f x) / (t : EReal)
```

Second stage (`RW_chap_3_26`).

`LeanWorkspace/Defs.lean`:

```
noncomputable def horizonFunction (f : E → EReal) (x : E) : EReal :=
  ⊓ (α : ℝ) (_ : (x, α) ∈ horizonCone (epigraph f)), (α : EReal)
```

`LeanWorkspace/RW_chap_3_26.lean`:

```
theorem RW_chap_3_26 {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    [FiniteDimensional ℝ E] (f : E → EReal)
    (hlsc : LowerSemicontinuous f) (hproper : IsProper f) :
    -- 3(7)
    (liminf (fun x => f x / (‖x‖ : EReal)) (cobounded E) =
        ⊓ (x : E) (_ : ‖x‖ = 1), horizonFunction f x) ∧
    ...
```

Claude Opus 5.5, Claude Code, attempt 1

First stage (`FCA_chap_B_3_2_4_ii`).

`LeanWorkspace/Defs.lean` contains only `import Mathlib`.

`LeanWorkspace/FCA_chap_B_3_2_4_ii.lean`:

```
theorem FCA_chap_B_3_2_4_ii {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    [FiniteDimensional ℝ E] (f : E → EReal)
    (hf_bot : ∀ x, f x ≠ ⊥)
    (hf_conv : Convex ℝ {p : E × ℝ | f p.1 ≤ p.2})
    (hf_lsc : LowerSemicontinuous f)
    (x₀ : E) (hx₀ : f x₀ ≠ ⊤) :
    List.TFAE
      [∃ r : ℝ, {x | f x ≤ r}.Nonempty ∧ IsCompact {x | f x ≤ r},
       ∀ r : ℝ, IsCompact {x | f x ≤ r},
       ∀ d : E, d ≠ 0 → 0 < ⨆ t > (0 : ℝ), (f (x₀ + t • d) - f x₀) / (t : EReal)] := by
  sorry
```

Second stage (`RW_chap_3_26`).

`LeanWorkspace/Defs.lean`:

```
namespace RW

noncomputable def horizonFunction {E : Type*} [AddCommGroup E] [Module ℝ E] [TopologicalSpace E]
    (f : E → EReal) : E → EReal :=
  if ∃ x, f x ≠ ⊤ then
    fun w => ⊓ (β : ℝ) (_ : (w, β) ∈ horizonCone {p : E × ℝ | f p.1 ≤ p.2}), (β : EReal)
  else
    fun w => if w = 0 then 0 else ⊤
```

`LeanWorkspace/RW_chap_3_26.lean`:

```
theorem RW_chap_3_26 {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    [FiniteDimensional ℝ E] (f : E → EReal)
    (hf_lsc : LowerSemicontinuous f)
    (hf_top : ∃ x, f x ≠ ⊤) (hf_bot : ∀ x, f x ≠ ⊥) :
    -- formula 3(7)
    Filter.liminf (fun x => f x / ((‖x‖ : ℝ) : EReal)) (Filter.comap (fun x : E => ‖x‖)
        Filter.atTop) = (⊓ (x : E) (_ : ‖x‖ = 1), RW.horizonFunction f x) ∧
    ...
```

Claude Opus 5.5, Claude Code, attempt 2

First stage (`FCA_chap_B_3_2_4_ii`).

`LeanWorkspace/Defs.lean` contains only `import Mathlib`.

`LeanWorkspace/FCA_chap_B_3_2_4_ii.lean`:

```
theorem FCA_chap_B_3_2_4_ii
    {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- `f ∈ \overline{Conv} E`: `f` takes values in `ℝ ∪ {+∞}`, is convex and closed
    (hf_ne_bot : ∀ x, f x ≠ ⊥)
    (hf_convex : ∀ x x' : E, ∀ α : ℝ, 0 < α → α < 1 →
      f (α • x + (1 - α) • x') ≤ (α : EReal) * f x + ((1 - α : ℝ) : EReal) * f x')
    (hf_closed : LowerSemicontinuous f)
    -- an arbitrary point `x₀ ∈ dom f`, used to compute the asymptotic function `f'_∞`
    (x₀ : E) (hx₀ : f x₀ ≠ ⊤) :
    List.TFAE
      [ ∃ r : ℝ, {x | f x ≤ r}.Nonempty ∧ IsCompact {x | f x ≤ r},
        ∀ r : ℝ, IsCompact {x | f x ≤ r},
        ∀ d : E, d ≠ 0 →
          0 < ⨆ (t : ℝ) (_ : 0 < t), (f (x₀ + t • d) - f x₀) / (t : EReal) ] := by
  sorry
```

Second stage (`RW_chap_3_26`).

`LeanWorkspace/Defs.lean`:

```
namespace VariationalAnalysis

noncomputable def horizonFunction [TopologicalSpace E] [AddCommGroup E] [Module ℝ E]
    (f : E → EReal) : E → EReal :=
  if ∀ x, f x = ⊤ then
    fun w => if w = 0 then 0 else ⊤
  else
    fun w => ⨅ (β : ℝ) (_ : (w, β) ∈ horizonCone (epigraph f)), (β : EReal)
```

`LeanWorkspace/RW_chap_3_26.lean`:

```
theorem RW_chap_3_26
    {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → EReal) (hf_lsc : LowerSemicontinuous f) (hf_proper : IsProper f) :
    -- formula 3(7)
    (liminf (fun x => f x / (‖x‖ : EReal)) (Bornology.cobounded E)
        = ⨅ (x : E) (_ : ‖x‖ = 1), horizonFunction f x) ∧
    ...
```

G.3.2 Detailed prompt

GPT-6 Astra, Codex, attempt 1

First stage (`FCA_chap_B_3_2_4_ii`).

`LeanWorkspace/Defs.lean`:

```
noncomputable def asymptoticFun {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    (f : E → EReal) : E → EReal :=
  funFromSet (asymptoticCone ℝ (epigraph f))
```

Second stage (`RW_chap_3_26`).

`LeanWorkspace/Defs.lean`: no new definitions for the shared notion.

`LeanWorkspace/RW_chap_3_26.lean`:

```
theorem RW_chap_3_26
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- Function: lower semicontinuous and proper.
    (hf_lsc : LowerSemicontinuous f)
    (hf_proper : IsProperEReal f) :
    -- Equation 3(7).
    (Filter.liminf (fun x : E => f x / (‖x‖ : EReal)) (Bornology.cobounded E) =
      sInf (asymptoticFun f '' {x : E | ‖x‖ = 1})) ∧
    ...
```

GPT-6 Astra, Codex, attempt 2

First stage (`FCA_chap_B_3_2_4_ii`).

`LeanWorkspace/Defs.lean`:

```
noncomputable def asymptoticFun {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    (f : E → EReal) : E → EReal :=
  funFromSet (horizonCone (epigraph f))
```

Second stage (`RW_chap_3_26`).

`LeanWorkspace/Defs.lean`: no new definitions for the shared notion.

`LeanWorkspace/RW_chap_3_26.lean`:

```
theorem RW_chap_3_26
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
    [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- Function: lower semicontinuous and proper.
    (hf_lowerSemicontinuous : LowerSemicontinuous f)
    (hf_proper : IsProper f) :
    -- Equation 3(7).
    (Filter.liminf (fun x => f x / (‖x‖ : EReal)) (Bornology.cobounded E) =
      sInf (asymptoticFun f '' {x : E | ‖x‖ = 1})) ∧
    ...
```

Claude Fable 5.1, Claude Code, attempt 1

First stage (`FCA_chap_B_3_2_4_ii`).

`LeanWorkspace/Defs.lean`:

```
def asymptoticFun {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] (f : E → EReal) :
    E → EReal :=
  funFromSet (horizonCone (epigraph f))
```

Second stage (`RW_chap_3_26`).

`LeanWorkspace/Defs.lean`: no new definitions for the shared notion.

`LeanWorkspace/RW_chap_3_26.lean`:

```
theorem RW_chap_3_26
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- `f : ℝ^n → ℝ` is lsc and proper
    (hf_lsc : LowerSemicontinuous f)
    (hf_proper : IsProperEReal f) :
    -- 3(7): `liminf_{|x|→∞} f(x)/|x| = inf_{|x|=1} f^∞(x)`
    Filter.liminf (fun x => f x / (‖x‖ : EReal)) (Bornology.cobounded E)
        = sInf (asymptoticFun f '' {x : E | ‖x‖ = 1}) ∧
    ...
```

Claude Fable 5.1, Claude Code, attempt 2

First stage (`FCA_chap_B_3_2_4_ii`).

`LeanWorkspace/Defs.lean`:

```
def asymptoticFun {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] (f : E → EReal) :
    E → EReal :=
  funFromSet (asymptoticCone ℝ (epigraph f))
```

Second stage (`RW_chap_3_26`).

`LeanWorkspace/Defs.lean`: no new definitions for the shared notion.

LeanWorkspace/RW_chap_3_26.lean:

```
theorem RW_chap_3_26
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- `f : ℝ^n → ℝ` is lsc and proper
    (hf_lsc : LowerSemicontinuous f)
    (hf_proper : IsProperEReal f) :
    -- 3(7): `lim inf_{|x|→∞} f(x)/|x| = inf_{|x|=1} f^∞(x)`
    liminf (linearGrowthQuotient f) (Bornology.cobounded E)
        = sInf (asymptoticFun f '' {x : E | ‖x‖ = 1}) ∧
    ...
```

Claude Opus 5.5, Claude Code, attempt 1

First stage (FCA_chap_B_3_2_4_ii).

LeanWorkspace/Defs.lean:

```
noncomputable def asymptoticFun {E : Type*} [AddCommGroup E] [Module ℝ E] [TopologicalSpace E]
    (f : E → EReal) : E → EReal :=
  funFromSet (horizonCone (epigraph f))
```

LeanWorkspace/FCA_chap_B_3_2_4_ii.lean:

```
theorem FCA_chap_B_3_2_4_ii
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- `f ∈ Conv̄ ℝ^n`: values in `ℝ ∪ {+∞}` and not identically `+∞`, convex, closed
    (hf_proper : (∀ x, ⊥ < f x) ∧ (effDom f).Nonempty)
    ...
```

Second stage (RW_chap_3_26).

LeanWorkspace/Defs.lean:

```
def ProperEReal (f : E → EReal) : Prop := (∀ x, ⊥ < f x) ∧ (effDom f).Nonempty
```

LeanWorkspace/FCA_chap_B_3_2_4_ii.lean (edited):

```
theorem FCA_chap_B_3_2_4_ii
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- `f ∈ Conv̄ ℝ^n`: values in `ℝ ∪ {+∞}` and not identically `+∞`, convex, closed
    (hf_proper : ProperEReal f)
    ...
```

LeanWorkspace/RW_chap_3_26.lean:

```
theorem RW_chap_3_26
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- `f` is lsc and proper
    (hf_lsc : LowerSemicontinuous f)
    (hf_proper : ProperEReal f) :
    -- 3(7): `liminf_{|x|→∞} f(x)/|x| = inf_{|x|=1} f^∞(x)`
    Filter.liminf (fun x => f x / (‖x‖ : EReal)) (Bornology.cobounded E) =
        sInf (asymptoticFun f '' Metric.sphere (0 : E) 1) ∧
    ...
```

Claude Opus 5.5, Claude Code, attempt 2

First stage (FCA_chap_B_3_2_4_ii).

LeanWorkspace/Defs.lean:

```
noncomputable def asymptoticFun (f : E → EReal) : E → EReal :=
  funFromSet (horizonCone (epigraph f))
```

Second stage (`RW_chap_3_26`).

`LeanWorkspace/Defs.lean`: no new definitions for the shared notion.

`LeanWorkspace/RW_chap_3_26.lean`:

```
theorem RW_chap_3_26
    {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
    (f : E → EReal)
    -- `f : ℝ^n → ℝ` is lsc and proper
    (hf_lsc : LowerSemicontinuous f)
    (hf_proper : ProperEReal f) :
    -- 3(7)
    liminf (fun x => f x / (‖x‖ : EReal)) (Bornology.cobounded E) =
        sInf (asymptoticFun f '' Metric.sphere (0 : E) 1) ∧
    ...
```

H Task instructions and prompts

H.1 Textbook tasks

H.1.1 Task instruction template

The fifteen minute and two hour evaluations share one instruction template and differ only in its header. The fifteen minute opening omits the time limit and the statement that the run continues until a submission is accepted. Each opening is followed by the shared information below, with `{lean_version}` set to v4.21.0 for the fifteen minute evaluation and v4.28.0 for the two hour evaluation.

Fifteen minute header.

```
# Prove `{theorem_name}`

You are given a Lean theorem with `sorry` in place of its proof. Produce a Lean proof of that
theorem.

When you have a complete proof, run `submit` to have it graded.
```

Two hour header.

```
# Prove `{theorem_name}`

You are given a Lean theorem with `sorry` in place of its proof. Produce a Lean proof of that
theorem.

**Time.** You have 2 hours for this task.

When you have a complete proof, run `submit` to have it graded. The run continues until a
submission is accepted or the time is used up.
```

Shared information.

```
## Problem statement

`/app/convex-eval/LeanWorkspace/{module}.lean` contains the definitions the statement depends
on, followed by the theorem `{theorem_name}`.

## Environment

- Lean {lean_version} with Mathlib is installed. The Lake package is `/app/convex-eval/`,
which holds `lakefile.toml`, `lean-toolchain`, `lake-manifest.json`, the library root
`LeanWorkspace.lean`, and the `LeanWorkspace/` module directory.
```

```
- Prebuilt Mathlib oleans are available.
- The sandbox has internet access.
- Check your progress from `/app/convex-eval/`:
  - `lake env lean LeanWorkspace/{module}.lean` - builds a single file and reports errors,
goal states at each `sorry`, and warnings.
  - `lake build LeanWorkspace` - what the grader runs.

## Scratch work

`/app/scratch/` is the scratch directory. Put everything that is not part of the submission
there: prose drafts, failed attempts, notes, calculations, throwaway scripts.

- It sits outside the Lake package, so nothing in it can affect the build.
- **None of it is graded.** A proof that only exists in `/app/scratch/` scores nothing - only
Lean code under `/app/convex-eval/` is graded.

## Save your work

Whatever is on disk when the run ends is your submission.

- After any non-trivial progress, save it back into the files under `LeanWorkspace/`.
- Leave your scratch work in `/app/scratch/`. It will not be graded.

## Success criteria

1. `lake build LeanWorkspace`, run from `/app/convex-eval/`, exits successfully.
2. `LeanWorkspace.lean`, `lakefile.toml`, `lean-toolchain` and `lake-manifest.json` are
unchanged, and so are the theorem statement and the given definitions.
3. `#print axioms` on the theorem you prove -- `{theorem_name}` -- reports no `sorryAx` and
no axiom beyond Lean's own `propext`, `Classical.choice`, and `Quot.sound`.
4. No `native_decide`.
5. Lean comparator accepts the proof.
```

H.2 Open problems

H.2.1 Task instruction template

```
# Prove `{theorem_name}`

You are given a Lean theorem and its negation with `sorry` in place of their proofs. Select
one version and produce a Lean proof of that theorem.

**Budget.** You have a budget of \${budget_usd} for this task.

When you have a complete proof, run `submit pos` or `submit neg` to have it graded. The run
continues until a submission is accepted or the budget is fully used.

## Problem statements

**Informal statement.** `/app/open-eval/statement.tex` states the problem in LaTeX.

**Formal statements.** `/app/open-eval/LeanWorkspace/{theorem_filename}_defs.lean` contains
the definitions in Lean. The files `/app/open-eval/LeanWorkspace/{theorem_filename}_pos.lean`
and `/app/open-eval/LeanWorkspace/{theorem_filename}_neg.lean` contain the Lean theorem and
its negative, respectively. They import the definitions file. Fill in exactly one of the two
`sorry`s and leave the other untouched; the unproven sibling theorem cannot be used in your
proof, since its `sorry` carries the `sorryAx` axiom, which the grader rejects.

## Environment

- Lean v4.28.0 with Mathlib is installed. The Lake package is `/app/open-eval/`,
which holds `lakefile.toml`, `lean-toolchain`, `lake-manifest.json`, the library root
`LeanWorkspace.lean`, and the `LeanWorkspace/` module directory.
- Prebuilt Mathlib oleans are available.
- The sandbox has internet access.
- Check your progress from `/app/open-eval/`:
```

```
  - `lake env lean LeanWorkspace/{theorem_filename}_pos.lean`, `lake env lean
LeanWorkspace/{theorem_filename}_neg.lean` - builds a single file and reports errors, goal
states at each `sorry`, and warnings.
  - `lake build LeanWorkspace` - what the grader runs.

## Scratch work

`/app/scratch/` is the scratch directory. Put everything that is not part of the submission
there: prose drafts, failed attempts, notes, calculations, throwaway scripts.

- It sits outside the Lake package, so nothing in it can affect the build.
- **None of it is graded.** A proof that only exists in `/app/scratch/` scores nothing - only
Lean code under `/app/open-eval/` is graded.

## Save your work

Whatever is on disk when the run ends is your submission.

- After any non-trivial progress, save it back into the files under `LeanWorkspace/`.
- Leave your scratch work in `/app/scratch/`. It will not be graded.

## Success criteria

1. `lake build LeanWorkspace`, run from `/app/open-eval/`, exits successfully.
2. `LeanWorkspace.lean`, `lakefile.toml`, `lean-toolchain` and `lake-manifest.json` are
unchanged, and so are the theorem statement and the given definitions.
3. `#print axioms` on the theorem you prove -- `{theorem_name}_POS` or `{theorem_name}_NEG`
-- reports no `sorryAx` and no axiom beyond Lean's own `propext`, `Classical.choice`, and
`Quot.sound`.
4. No `native_decide`.
5. Lean comparator accepts the proof.

Judgements about whether the problem is open are not accepted as solutions.
```

H.2.2 Agent prompt

The agent prompt accompanies the task instruction.

```
{{ instruction }}
**This may be an open conjecture, and that does not make it impossible.** In 2026, systems
like you have settled problems open for decades. The planar unit-distance problem,
posed by Erdős in 1946, was resolved with a machine-checked Lean proof. For the forced
three-dimensional Navier-Stokes equations, a finite-time singularity was proved, again
formalized in Lean. Large known results, such as Fermat's Last Theorem, have been fully
formalized. The difficulty is real, but it is not a wall. You are miscalibrated: your
instinct will be to decide the problem is hopeless, or "already open," and stop. Here that
instinct is wrong.

**Do not stop early - this is the one way these attempts reliably fail.** Deciding the
problem is open, or that it reduces to another hard statement, is not a solution and not
a stopping point. Engage the problem itself: prove intermediate and partial results, test
concrete approaches, and work through the actual difficulty. Do not merely survey what is
known about it. A proof is rarely in hand in the first hours. The budget exists precisely so
you can keep trying genuine attempts long past your first urge to quit. And do not refuse a
line of work because you judge it too expensive to finish within the budget. Such estimates,
of difficulty and of cost alike, are unreliable and have proven too pessimistic before, so
begin the work and get as far as the budget takes you.

**How to approach it is yours to decide.** Bring whatever mathematics the problem calls
for. Two things are worth keeping in mind. First, it is not known in advance whether the
statement is true, so trying to disprove it can be as productive as trying to prove it -
the Navier-Stokes singularity above was found only because some groups pursued the negation.
Pursue both directions. But blindly searching for counterexamples with numerics is rarely a
good approach: reserve it for checking specific candidates your reasoning has singled out.
Second, try genuinely different approaches rather than betting everything on one. Pursue
each far enough to genuinely test it rather than merely naming it, then set aside those that
stall and carry the promising ones to a finish. This breadth of approach is your particular
```

```
strength: over the course of your work you can attempt far more genuinely different lines
than any single researcher could.

**The goal, and where work goes.** The grader's verdict is binary: it accepts a complete
Lean proof of the theorem or its negation and scores nothing else. That accepted proof is
the goal. Formalize what you establish: for the grade, a result exists only as compiled,
sorry-free Lean under `/app/open-eval/`. Keep those files clean. Importing a file that still
contains a `sorry` makes `#print axioms` report `sorryAx` and the whole proof is rejected,
so put only the proof and the lemmas it uses there. Everything else - exploration, failed
attempts, notes, searches - belongs in `/app/scratch/`, which is not graded and cannot
break your build. Formalization only records a result you already have, so it is not where
the budget should go. Spend it on the mathematics - genuinely attempting many different
approaches - and when a line stalls, open a different one rather than retreating into routine
Lean work or infrastructure the proof does not need.

**Important**: Keep a bounded number of subagents and background processes running at once.
Subagents must write their findings to files in `/app/scratch/` and return only a brief
summary and the file path. They must not return full transcripts or large outputs into the
main context. Have subagents ping the main agent at intervals so it can intervene. Ending
your turn terminates any command started with `run_in_background`.
```

H.2.3 Agent continuation message

Whenever the agent ends a trial but the submission is not successful and time/budget remains, then the trial is resumed with the following continuation message:

```
You have permission to keep working on this problem, and there is no need to conserve the
remaining budget.
```

I List of formalized textbook statements

Table 5: Sources of the 315 textbook results, grouped by the reference they were formalized from. Each Lean theorem is listed with its numbered result, or the displayed equation where the result is unnumbered. Theorems marked * are the 100 textbook task subset used for the fifteen minute evaluation.

Lean theorem	Result
Convex Analysis and Minimization Algorithms I	
CAMA_chap_3_1_2_1*	Chapter III, Proposition 1.2.1
CAMA_chap_3_1_2_4*	Chapter III, Proposition 1.2.4
CAMA_chap_3_1_2_7	Chapter III, Proposition 1.2.7
CAMA_chap_3_1_3_3	Chapter III, Proposition 1.3.3
CAMA_chap_3_1_3_4	Chapter III, Proposition 1.3.4
CAMA_chap_3_1_3_6	Chapter III, Theorem 1.3.6 (C. Carathéodory)
CAMA_chap_3_1_4_3*	Chapter III, Theorem 1.4.3
CAMA_chap_3_2_1_3	Chapter III, Theorem 2.1.3
CAMA_chap_3_2_1_6*	Chapter III, Lemma 2.1.6
CAMA_chap_3_2_1_8	Chapter III, Proposition 2.1.8
CAMA_chap_3_2_1_10	Chapter III, Proposition 2.1.10
CAMA_chap_3_2_1_12	Chapter III, Proposition 2.1.12
CAMA_chap_3_2_2_1	Chapter III, Proposition 2.2.1
CAMA_chap_3_2_2_3	Chapter III, Proposition 2.2.3
CAMA_chap_3_2_3_3*	Chapter III, Proposition 2.3.3
CAMA_chap_3_2_3_4*	Chapter III, Theorem 2.3.4 (H. Minkowski)
CAMA_chap_3_2_3_7*	Chapter III, Proposition 2.3.7
CAMA_chap_3_2_4_6	Chapter III, Proposition 2.4.6
CAMA_chap_3_3_1_1*	Chapter III, Theorem 3.1.1
CAMA_chap_3_3_1_3	Chapter III, Proposition 3.1.3
CAMA_chap_3_3_2_3	Chapter III, Proposition 3.2.3
CAMA_chap_3_3_2_5	Chapter III, Theorem 3.2.5
CAMA_chap_3_4_1_1	Chapter III, Theorem 4.1.1
CAMA_chap_3_4_1_3*	Chapter III, Corollary 4.1.3
CAMA_chap_3_4_2_1	Chapter III, Lemma 4.2.1

Continued on next page

Table 5 continued

Lean theorem	Result
`CAMA_chap_3_4_2_3`	Chapter III, Proposition 4.2.3
`CAMA_chap_3_4_2_4`*	Chapter III, Theorem 4.2.4
`CAMA_chap_3_4_2_7`	Chapter III, Proposition 4.2.7
`CAMA_chap_3_4_3_1`	Chapter III, Lemma 4.3.1
`CAMA_chap_3_4_3_2`	Chapter III, Lemma 4.3.2
`CAMA_chap_3_4_3_3`	Chapter III, Lemma 4.3.3
`CAMA_chap_3_5_1_2`*	Chapter III, Proposition 5.1.2
`CAMA_chap_3_5_1_3`*	Chapter III, Proposition 5.1.3
`CAMA_chap_3_5_2_1`	Chapter III, Proposition 5.2.1
`CAMA_chap_3_5_2_4`*	Chapter III, Proposition 5.2.4
`CAMA_chap_3_5_2_5`	Chapter III, Corollary 5.2.5
`CAMA_chap_3_5_3_1`	Chapter III, Proposition 5.3.1
`CAMA_chap_3_5_3_3`	Chapter III, Proposition 5.3.3
Fundamentals of Convex Analysis	
`FCA_chap_A_ex_2`	Chapter A, Exercise 2
`FCA_chap_A_ex_3`	Chapter A, Exercise 3
`FCA_chap_A_ex_5`	Chapter A, Exercise 5
`FCA_chap_A_ex_8`	Chapter A, Exercise 8
`FCA_chap_A_ex_11`*	Chapter A, Exercise 11
`FCA_chap_A_ex_12`*	Chapter A, Exercise 12
`FCA_chap_A_ex_13`	Chapter A, Exercise 13
`FCA_chap_A_ex_14`	Chapter A, Exercise 14
`FCA_chap_A_ex_15`	Chapter A, Exercise 15
`FCA_chap_A_ex_18`*	Chapter A, Exercise 18
`FCA_chap_A_ex_20`	Chapter A, Exercise 20
`FCA_chap_A_ex_21`*	Chapter A, Exercise 21
`FCA_chap_A_ex_22`	Chapter A, Exercise 22
`FCA_chap_A_ex_23`	Chapter A, Exercise 23
`FCA_chap_A_ex_26`	Chapter A, Exercise 26
`FCA_chap_B_1_1_2`*	Chapter B, Proposition 1.1.2
`FCA_chap_B_1_1_6`	Chapter B, Proposition 1.1.6
`FCA_chap_B_1_1_8`	Chapter B, Theorem 1.1.8 (Inequality of Jensen)
`FCA_chap_B_1_1_9`	Chapter B, Proposition 1.1.9
`FCA_chap_B_1_2_1`*	Chapter B, Proposition 1.2.1
`FCA_chap_B_1_2_2`*	Chapter B, Proposition 1.2.2
`FCA_chap_B_1_2_5`*	Chapter B, Proposition 1.2.5
`FCA_chap_B_1_2_6`	Chapter B, Proposition 1.2.6
`FCA_chap_B_1_2_8`	Chapter B, Proposition 1.2.8
`FCA_chap_B_1_3_1`	Chapter B, Theorem 1.3.1
`FCA_chap_B_2_1_1`	Chapter B, Proposition 2.1.1
`FCA_chap_B_2_1_2`	Chapter B, Proposition 2.1.2
`FCA_chap_B_2_1_4`*	Chapter B, Proposition 2.1.4
`FCA_chap_B_2_2_1`	Chapter B, Proposition 2.2.1
`FCA_chap_B_2_2_2`*	Chapter B, Proposition 2.2.2
`FCA_chap_B_2_3_3`	Chapter B, Proposition 2.3.3
`FCA_chap_B_2_4_2`	Chapter B, Theorem 2.4.2
`FCA_chap_B_2_4_3`	Chapter B, Corollary 2.4.3
`FCA_chap_B_2_4_5`	Chapter B, Corollary 2.4.5
`FCA_chap_B_2_5_1`	Chapter B, Proposition 2.5.1
`FCA_chap_B_2_5_2`	Chapter B, Proposition 2.5.2
`FCA_chap_B_2_5_4`	Chapter B, Proposition 2.5.4
`FCA_chap_B_3_1_1`	Chapter B, Lemma 3.1.1
`FCA_chap_B_3_1_2`	Chapter B, Theorem 3.1.2
`FCA_chap_B_3_1_4`	Chapter B, Theorem 3.1.4
`FCA_chap_B_3_2_1`	Chapter B, Proposition 3.2.1
`FCA_chap_B_3_2_4_i`*	Chapter B, Proposition 3.2.4, first part
`FCA_chap_B_3_2_4_ii`*	Chapter B, Proposition 3.2.4, second part
`FCA_chap_B_3_2_6`*	Chapter B, Proposition 3.2.6
`FCA_chap_B_3_2_8_i`	Chapter B, Proposition 3.2.8, first item
`FCA_chap_B_3_2_8_ii`	Chapter B, Proposition 3.2.8, second item
`FCA_chap_B_3_2_8_iii`*	Chapter B, Proposition 3.2.8, third item
`FCA_chap_B_4_1_1`*	Chapter B, Theorem 4.1.1
`FCA_chap_B_4_1_4`	Chapter B, Theorem 4.1.4

Continued on next page

Table 5 continued

Lean theorem	Result
FCA_chap_B_4_2_1	Chapter B, Proposition 4.2.1
FCA_chap_B_4_2_3	Chapter B, Theorem 4.2.3
FCA_chap_B_ex_19	Chapter B, Exercise 19
FCA_chap_B_ex_21	Chapter B, Exercise 21
FCA_chap_B_ex_24	Chapter B, Exercise 24
FCA_chap_B_ex_26	Chapter B, Exercise 26
FCA_chap_C_1_1_3*	Chapter C, Proposition 1.1.3
FCA_chap_C_1_1_4*	Chapter C, Proposition 1.1.4
FCA_chap_C_1_1_5*	Chapter C, Corollary 1.1.5
FCA_chap_C_1_1_6	Chapter C, Proposition 1.1.6
FCA_chap_C_1_1_7	Chapter C, Proposition 1.1.7
FCA_chap_C_1_2_5	Chapter C, Theorem 1.2.5
FCA_chap_C_1_2_6	Chapter C, Corollary 1.2.6
FCA_chap_C_1_3_1_i*	Chapter C, Proposition 1.3.1(i)
FCA_chap_C_1_3_1_ii*	Chapter C, Proposition 1.3.1(ii)
FCA_chap_C_1_3_2_i	Chapter C, Proposition 1.3.2(i)
FCA_chap_C_1_3_2_ii	Chapter C, Proposition 1.3.2(ii)
FCA_chap_C_1_3_5*	Chapter C, Theorem 1.3.5
FCA_chap_C_2_1_2*	Chapter C, Proposition 2.1.2
FCA_chap_C_2_1_3	Chapter C, Proposition 2.1.3
FCA_chap_C_2_2_1	Chapter C, Proposition 2.2.1
FCA_chap_C_2_2_2	Chapter C, Theorem 2.2.2
FCA_chap_C_2_2_3	Chapter C, Theorem 2.2.3
FCA_chap_C_2_2_4	Chapter C, Proposition 2.2.4
FCA_chap_C_3_1_1*	Chapter C, Theorem 3.1.1
FCA_chap_C_3_1_2	Chapter C, Corollary 3.1.2
FCA_chap_C_3_1_4	Chapter C, Proposition 3.1.4
FCA_chap_C_3_2_4	Chapter C, Proposition 3.2.4
FCA_chap_C_3_2_5	Chapter C, Corollary 3.2.5
FCA_chap_C_3_2_7*	Chapter C, Proposition 3.2.7
FCA_chap_C_3_3_1	Chapter C, Theorem 3.3.1
FCA_chap_C_3_3_2_i	Chapter C, Theorem 3.3.2(i)
FCA_chap_C_3_3_2_ii	Chapter C, Theorem 3.3.2(ii)
FCA_chap_C_3_3_3	Chapter C, Proposition 3.3.3
FCA_chap_C_3_3_4	Chapter C, Proposition 3.3.4
FCA_chap_C_ex1*	Chapter C, Exercise 1
FCA_chap_C_ex2	Chapter C, Exercise 2
FCA_chap_C_ex3*	Chapter C, Exercise 3
FCA_chap_C_ex4	Chapter C, Exercise 4
FCA_chap_C_ex5	Chapter C, Exercise 5
FCA_chap_C_ex10*	Chapter C, Exercise 10
FCA_chap_C_ex14	Chapter C, Exercise 14
FCA_chap_D_1_1_2	Chapter D, Proposition 1.1.2
FCA_chap_D_1_1_6*	Chapter D, Proposition 1.1.6
FCA_chap_D_1_2_2	Chapter D, Theorem 1.2.2
FCA_chap_D_1_3_1	Chapter D, Proposition 1.3.1
FCA_chap_D_1_3_2	Chapter D, Lemma 1.3.2
FCA_chap_D_1_3_3*	Chapter D, Proposition 1.3.3
FCA_chap_D_1_3_4	Chapter D, Theorem 1.3.4
FCA_chap_D_1_3_5*	Chapter D, Theorem 1.3.5
FCA_chap_D_2_1_1*	Chapter D, Lemma 2.1.1
FCA_chap_D_2_1_5	Chapter D, Proposition 2.1.5
FCA_chap_D_2_2_1*	Chapter D, Theorem 2.2.1
FCA_chap_D_2_3_1	Chapter D, Lemma 2.3.1
FCA_chap_D_2_3_3	Chapter D, Theorem 2.3.3
FCA_chap_D_2_3_4	Chapter D, Theorem 2.3.4
FCA_chap_D_4_1_1	Chapter D, Theorem 4.1.1
FCA_chap_D_4_2_1*	Chapter D, Theorem 4.2.1
FCA_chap_D_4_3_1	Chapter D, Theorem 4.3.1
FCA_chap_D_4_3_2*	Chapter D, Corollary 4.3.2
FCA_chap_D_4_4_1	Chapter D, Lemma 4.4.1
FCA_chap_D_4_4_2	Chapter D, Theorem 4.4.2
FCA_chap_D_4_4_4	Chapter D, Corollary 4.4.4
FCA_chap_D_4_4_5	Chapter D, Corollary 4.4.5

Continued on next page

Table 5 continued

Lean theorem	Result
FCA_chap_D_4_5_1	Chapter D, Theorem 4.5.1
FCA_chap_D_4_5_2	Chapter D, Corollary 4.5.2
FCA_chap_D_4_5_5	Chapter D, Corollary 4.5.5
FCA_chap_D_6_1_1	Chapter D, Proposition 6.1.1
FCA_chap_D_6_1_2	Chapter D, Theorem 6.1.2
FCA_chap_D_6_1_3	Chapter D, Proposition 6.1.3
FCA_chap_D_6_2_1⋆	Chapter D, Proposition 6.2.1
FCA_chap_D_6_2_2	Chapter D, Proposition 6.2.2
FCA_chap_D_6_2_4	Chapter D, Theorem 6.2.4
FCA_chap_D_6_2_5	Chapter D, Corollary 6.2.5
FCA_chap_D_6_2_7	Chapter D, Theorem 6.2.7
FCA_chap_D_6_2_8	Chapter D, Corollary 6.2.8
FCA_chap_D_6_3_1	Chapter D, Theorem 6.3.1
FCA_chap_D_ex2	Chapter D, Exercise 2
FCA_chap_D_ex5	Chapter D, Exercise 5
FCA_chap_D_ex6⋆	Chapter D, Exercise 6
FCA_chap_D_ex7	Chapter D, Exercise 7
FCA_chap_D_ex11	Chapter D, Exercise 11
FCA_chap_D_ex12	Chapter D, Exercise 12
FCA_chap_D_ex15	Chapter D, Exercise 15
FCA_chap_D_ex18⋆	Chapter D, Exercise 18
FCA_chap_E_1_1_2	Chapter E, Theorem 1.1.2
FCA_chap_E_1_2_1	Chapter E, Proposition 1.2.1
FCA_chap_E_1_3_1_i	Chapter E, Proposition 1.3.1(i)
FCA_chap_E_1_3_1_ii	Chapter E, Proposition 1.3.1(ii)
FCA_chap_E_1_3_1_iii	Chapter E, Proposition 1.3.1(iii)
FCA_chap_E_1_3_1_iv	Chapter E, Proposition 1.3.1(iv)
FCA_chap_E_1_3_1_v⋆	Chapter E, Proposition 1.3.1(v)
FCA_chap_E_1_3_1_vi	Chapter E, Proposition 1.3.1(vi)
FCA_chap_E_1_3_1_vii⋆	Chapter E, Proposition 1.3.1(vii)
FCA_chap_E_1_3_1_viii	Chapter E, Proposition 1.3.1(viii)
FCA_chap_E_1_3_2	Chapter E, Proposition 1.3.2
FCA_chap_E_1_3_4	Chapter E, Proposition 1.3.4
FCA_chap_E_1_3_5	Chapter E, Theorem 1.3.5
FCA_chap_E_1_3_6	Chapter E, Corollary 1.3.6
FCA_chap_E_1_3_8	Chapter E, Proposition 1.3.8
FCA_chap_E_1_3_9	Chapter E, Proposition 1.3.9
FCA_chap_E_1_4_1⋆	Chapter E, Theorem 1.4.1
FCA_chap_E_1_4_2	Chapter E, Theorem 1.4.2
FCA_chap_E_1_4_3	Chapter E, Proposition 1.4.3
FCA_chap_E_1_4_4	Chapter E, Corollary 1.4.4
FCA_chap_E_2_1_1	Chapter E, Theorem 2.1.1
FCA_chap_E_2_2_1	Chapter E, Theorem 2.2.1
FCA_chap_E_2_2_2	Chapter E, Lemma 2.2.2
FCA_chap_E_2_2_3	Chapter E, Theorem 2.2.3
FCA_chap_E_2_3_1	Chapter E, Theorem 2.3.1
FCA_chap_E_2_3_2⋆	Chapter E, Theorem 2.3.2
FCA_chap_E_3_3_1	Chapter E, Proposition 3.3.1
FCA_chap_E_4_1_1	Chapter E, Theorem 4.1.1
FCA_chap_E_4_1_2	Chapter E, Theorem 4.1.2
FCA_chap_E_4_1_3	Chapter E, Corollary 4.1.3
FCA_chap_E_4_2_1	Chapter E, Theorem 4.2.1
FCA_chap_E_4_2_2⋆	Chapter E, Theorem 4.2.2
FCA_chap_E_ex2	Chapter E, Exercise 2
FCA_chap_E_ex4	Chapter E, Exercise 4
FCA_chap_E_ex5	Chapter E, Exercise 5
FCA_chap_E_ex6⋆	Chapter E, Exercise 6
FCA_chap_E_ex7⋆	Chapter E, Exercise 7
FCA_chap_E_ex9⋆	Chapter E, Exercise 9
FCA_chap_E_ex15	Chapter E, Exercise 15
EE364M, Duchi 2024	
Duchi_L2_3_1	Exercise Set 2, Lemma 2.3.1
Duchi_L4_1_2⋆	Exercise Set 4, Lemma 4.1.2

Continued on next page

Table 5 continued

Lean theorem	Result
Duchi_L4_1_3	Exercise Set 4, Lemma 4.1.3
Duchi_L8_1_1	Exercise Set 8, Lemma 8.1.1
Duchi_P4_1_1	Exercise Set 4, Proposition 4.1.1
Duchi_Q1_1⋆	Exercise Set 1, Question 1.1
Duchi_Q2_3	Exercise Set 2, Question 2.3
Duchi_Q5_1	Exercise Set 5, Question 5.1
Duchi_Q5_2a⋆	Exercise Set 5, Question 5.2(a)
Duchi_Q5_2b⋆	Exercise Set 5, Question 5.2(b)
Duchi_Q5_2c	Exercise Set 5, Question 5.2(c)
Duchi_Q9_1⋆	Exercise Set 9, Question 9.1
Duchi_T10_2_1	Exercise Set 10, Theorem 10.2.1
Convex Optimization: Algorithms and Complexity	
bubeck_3_13	Theorem 3.13
bubeck_3_14⋆	Theorem 3.14
center_of_gravity_bound⋆	Theorem 2.1
dual_averaging_bound⋆	Theorem 4.3
ellipsoid_method_bound	Theorem 2.4
FISTA_bound⋆	Unnumbered display, Section 5.1 (FISTA), p. 311
frank_wolfe_smooth_bound	Theorem 3.8
gradient_descent_smooth_bound	Theorem 3.3
gradient_descent_smooth_strongly_convex_bound	Theorem 3.12
grunbaum⋆	Lemma 2.2 (Grünbaum)
ipm_path_following_bound_centered⋆	Theorem 5.6
ISTA_bound	Unnumbered display after (5.1), Section 5.1, p. 310
minibatch_stochastic_gradient_descent_bound⋆	Unnumbered display, Section 6.2 (mini-batch SGD), p. 334
mirror_descent_bound	Theorem 4.2
mirror_prox_bound	Theorem 4.4
nesterov_smooth_convex_bound⋆	Theorem 3.19
nesterov_smooth_strongly_convex_bound	Theorem 3.18
newton_decrement_bound⋆	Theorem 5.4
newtons_method_bound	Theorem 5.3
projected_gradient_descent_smooth_bound⋆	Theorem 3.7
projected_gradient_descent_smooth_strongly_convex_bound	Theorem 3.10
randomized_coordinate_descent_bound	Theorem 6.6
randomized_coordinate_descent_gamma_bound⋆	Theorem 6.7
randomized_coordinate_descent_gamma_stronglyconvex_bound	Theorem 6.8
saddle_point_mirror_descent_bound	Theorem 5.1
saddle_point_mirror_prox_bound⋆	Theorem 5.2
stochastic_gradient_descent_bound⋆	Theorem 6.2
stochastic_mirror_descent_bound	Theorem 6.1
stochastic_mirror_descent_bound_smooth	Theorem 6.3
subgradient_bound	Theorem 3.2
subgradient_strongly_convex_bound	Theorem 3.9
svrg_bound	Theorem 6.5
Schmidt et al. [2017], as stated in Tibshirani's 10-725 slides	
sag_bound_centered⋆	SAG convergence theorem, convex case
sag_strongly_convex_linear_bound⋆	SAG convergence under strong convexity
Defazio et al. [2014]	
saga_convex_average_bound⋆	Theorem 2, Appendix C
saga_strongly_convex_distance_bound⋆	Corollary 1
A mathematical view of interior-point methods in convex optimization	
Renegar_chap_2_2_2_9⋆	Theorem 2.2.9
Renegar_chap_2_2_2_10⋆	Proposition 2.2.10
Renegar_chap_2_2_3_1⋆	Theorem 2.3.1
Renegar_chap_2_2_3_3⋆	Theorem 2.3.3
Renegar_chap_2_2_3_4	Theorem 2.3.4
Renegar_chap_2_2_3_5	Corollary 2.3.5
Renegar_chap_2_2_3_6⋆	Corollary 2.3.6
Renegar_chap_2_2_3_8	Theorem 2.3.8

Continued on next page

Table 5 continued

Lean theorem	Result
`Renegar_chap_2_2_3_9_i`	Theorem 2.3.9, first part
`Renegar_chap_2_2_3_9_ii`*	Theorem 2.3.9, second part
`Renegar_chap_2_2_5_1`	Theorem 2.5.1
Variational Analysis	
`RW_chap_8_11`*	Corollary 8.11
`RW_chap_8_12`	Proposition 8.12
`RW_chap_8_15`*	Theorem 8.15
`RW_chap_8_18`	Theorem 8.18
`RW_chap_8_21`	Proposition 8.21
`RW_chap_8_22`*	Theorem 8.22
`RW_chap_8_24`	Theorem 8.24
`RW_chap_8_25`	Corollary 8.25
`RW_chap_8_29`	Proposition 8.29
`RW_chap_8_30`*	Theorem 8.30
`RW_chap_8_32`*	Proposition 8.32
`RW_chap_9_10`	Proposition 9.10
`RW_chap_9_12`	Exercise 9.12
`RW_chap_9_13`*	Theorem 9.13
`RW_chap_9_16`*	Theorem 9.16
`RW_chap_9_19`	Corollary 9.19
`RW_chap_11_8`	Theorem 11.8
`RW_chap_11_14`*	Theorem 11.14
`RW_chap_11_15`	Lemma 11.15
`RW_chap_11_17`*	Corollary 11.17
`RW_chap_11_34`	Theorem 11.34
`RW_chap_11_36`	Theorem 11.36
`RW_chap_12_15`*	Theorem 12.15
`RW_chap_12_16`	Exercise 12.16
`RW_chap_12_17`	Theorem 12.17
`RW_chap_12_18`	Corollary 12.18
`RW_chap_12_19`*	Proposition 12.19
`RW_chap_12_20`*	Corollary 12.20
`RW_chap_12_32`	Theorem 12.32
`RW_chap_12_35`*	Theorem 12.35
`RW_chap_12_37`*	Theorem 12.37
`RW_chap_12_38`	Corollary 12.38
`RW_chap_12_41`	Theorem 12.41
Convex Analysis and Monotone Operator Theory in Hilbert Spaces	
`BC_chap_6_30`	Theorem 6.30 (Moreau)
`BC_chap_7_12`	Corollary 7.12
`BC_chap_8_38`	Theorem 8.38
`BC_chap_9_20`	Theorem 9.20
`BC_chap_13_37`	Theorem 13.37
`BC_chap_16_17`	Proposition 16.17
`BC_chap_20_25`	Theorem 20.25
Classical results	
`brunn_minkowski`	Brunn–Minkowski inequality
`brunn_minkowski_1d`	Brunn–Minkowski inequality, one-dimensional case
`brunn_minkowski_multiplicative`	Brunn–Minkowski inequality, multiplicative form
`prekopa_leindler_1d`	Prékopa–Leindler inequality, one-dimensional case